

先進計算機構成論 07

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

shioya@ci.i.u-tokyo.ac.jp

メモリ

- メモリ：RAM（Random Access Memory）
 - 複数のデータを記憶する回路
 - 配列のように、位置を指定して読み書きする
- なぜデータを記憶するための専用の構造が使われているのか？
 - D-FF とマルチプレクサでも同じ機能のものは作れる
 - メモリの方が圧倒的に速度や面積効率が高い
 - 後半で説明
- なぜメモリの構造を勉強するのか？：その性質の理解が重要だから
 - 高度な CPU はメモリで作られた「表」を多用する
 - 主記憶やキャッシュなどは直接メモリで作られる

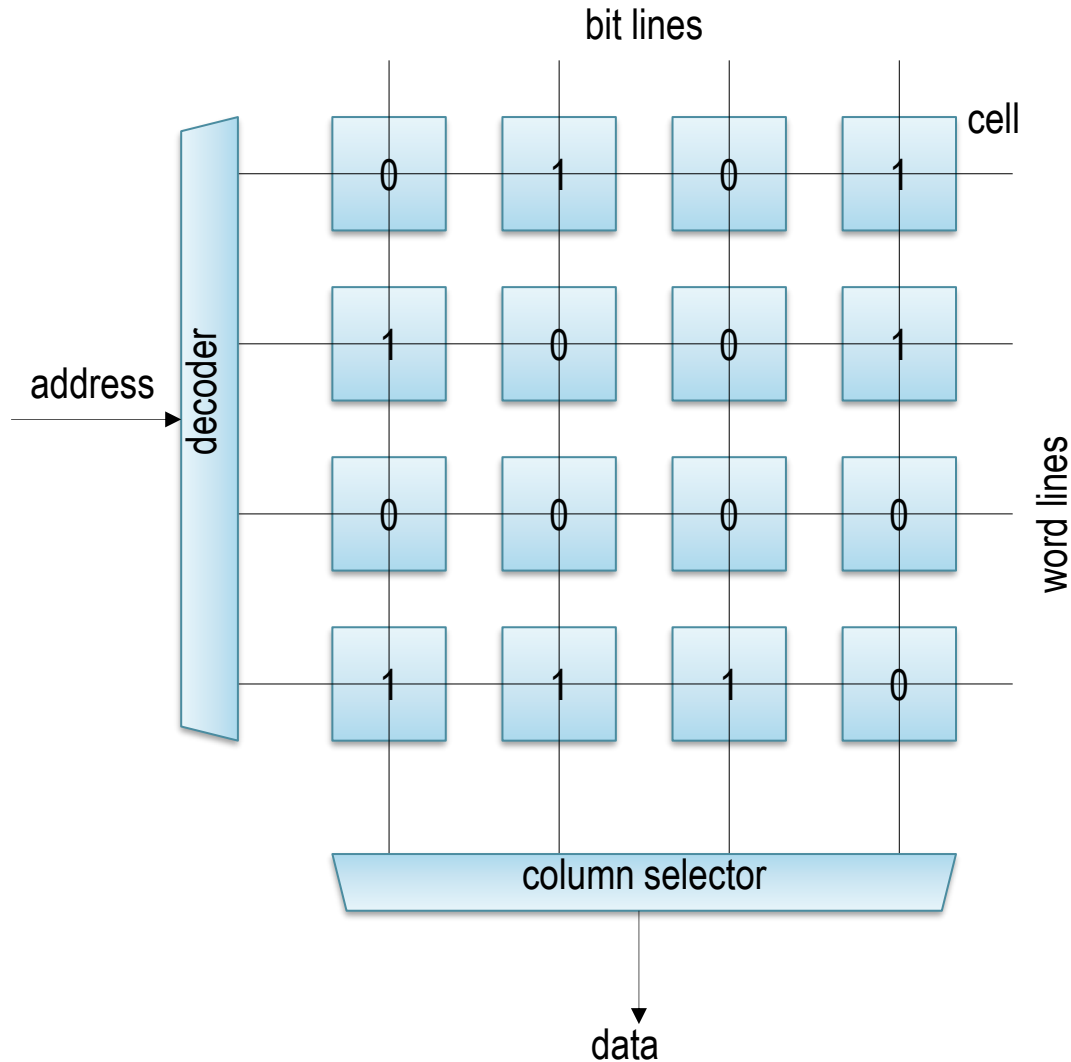
1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

2. メモリの詳細

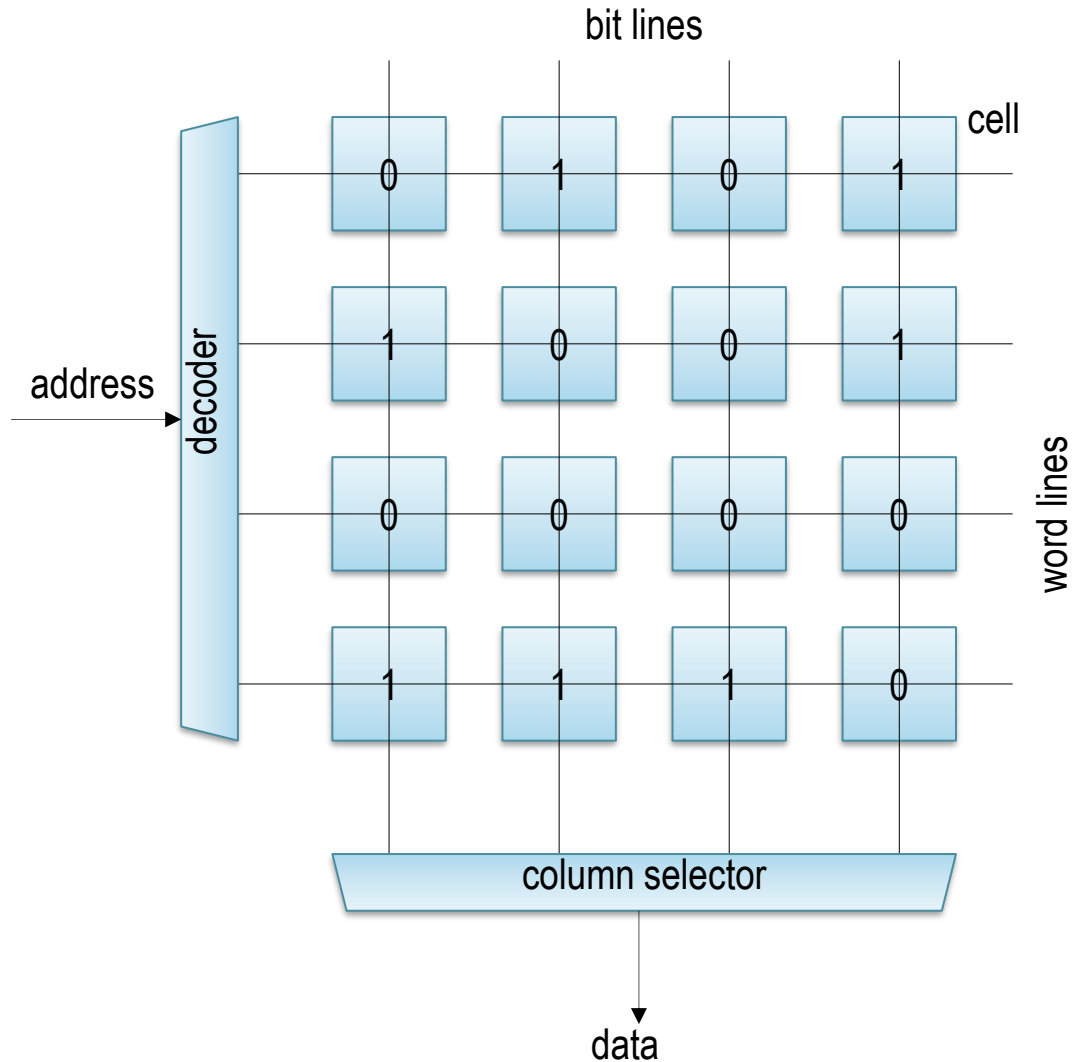
1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリの脆弱性を対象にしたアタック
 - Raw Hammer, Cold Boot Attack

メモリの基本構造：セルを行列状に配置



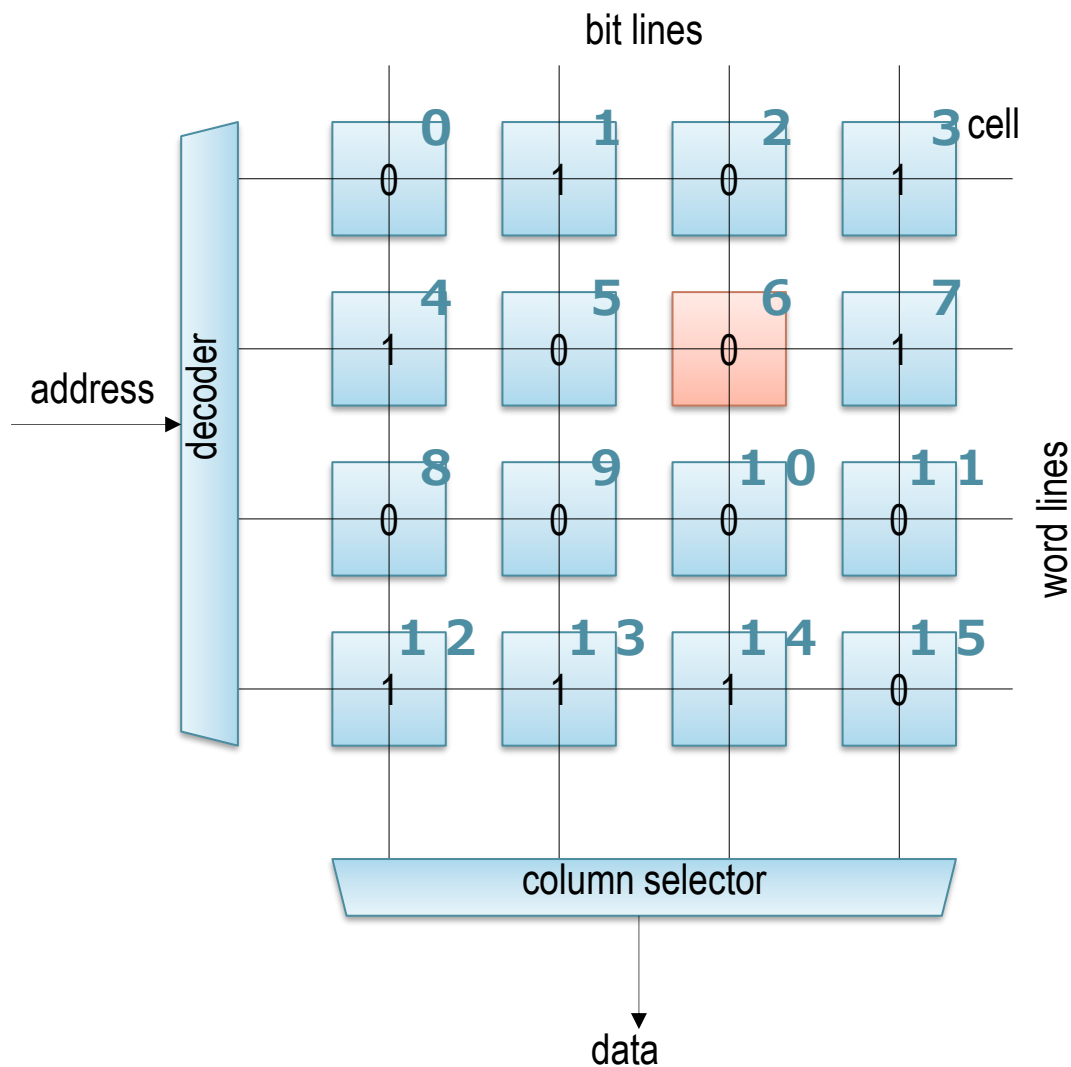
- セル：
 - 1ビットの情報を記憶
- ビットライン：
 - 列方向の配線
- ワードライン：
 - 行方向の配線
- デコーダ：
 - アドレスをデコードしてワードラインをアサート
- カラム・セクタ：
 - ビットライン出力から1つを選んで出力

メモリの読み出し操作



1. アドレスのデコード
 2. ワードラインのアサート (行の指定)
 3. ビットラインへの1行分のデータの読み出し
 4. カラムセクタによる目的のビットの選択
- (書き込みはこれの逆を行う)

メモリの読み出し動作の例



■ 要素数 $16 = 2^4$

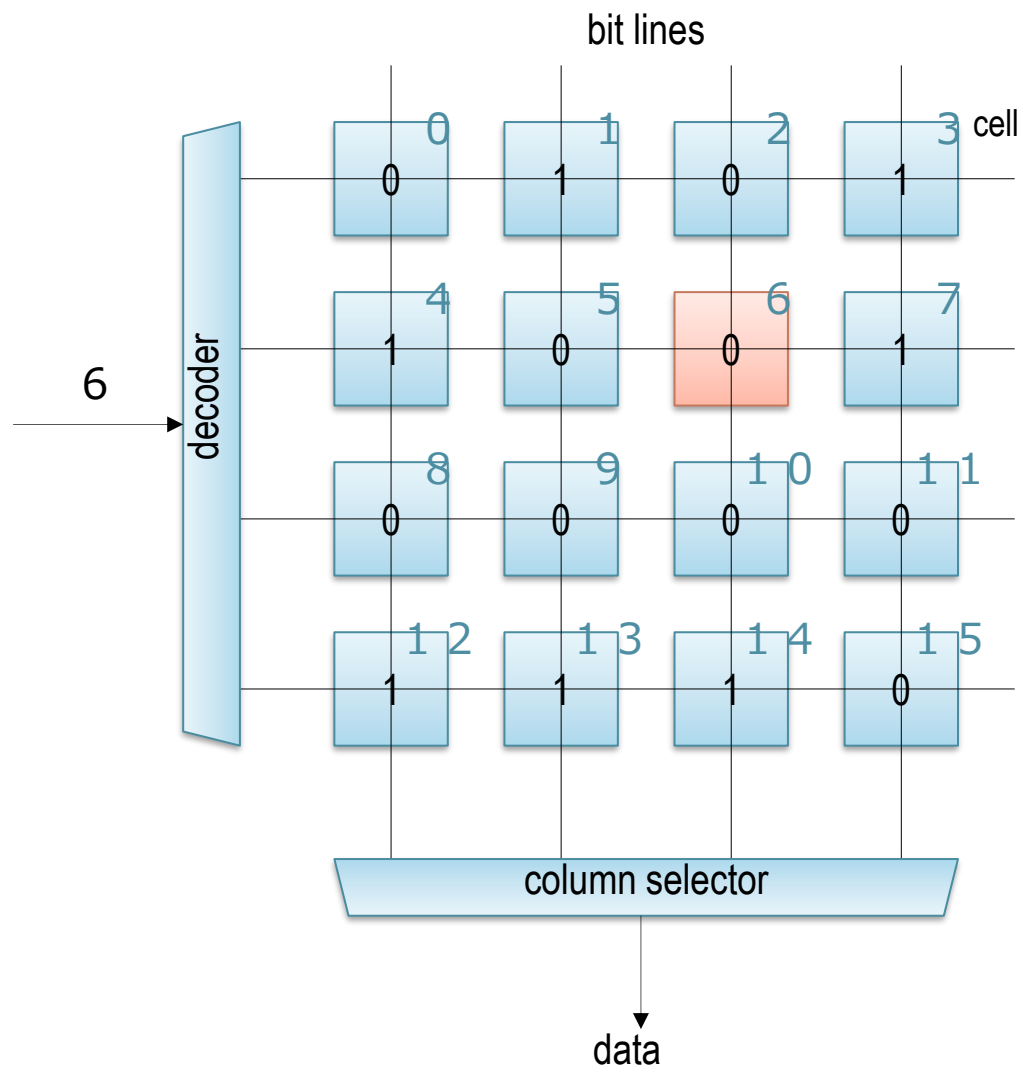
- アドレスは4ビット
- 各セルの右上の数字がアドレス

■ このメモリの読み出し操作を例として説明

- アドレス6のセルを読み出す

メモリの読み出し動作 (1)

アドレスのデコード



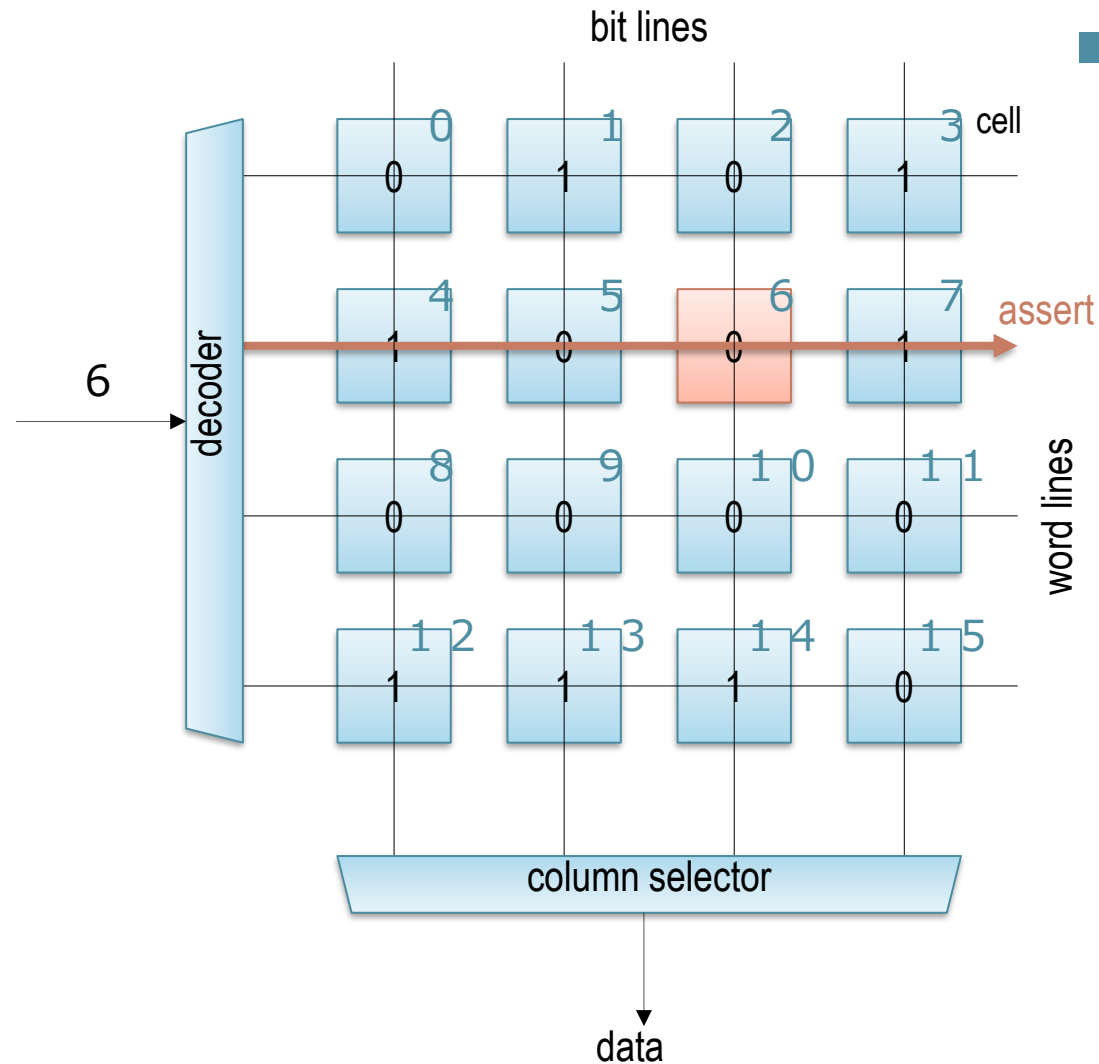
■ どの行を読むか決める

- 各行は4つセルがある
- アドレスを4で割って切り捨てた行目が読むべき場所
- $6 = 0110$ (2進数)

■ デコーダ

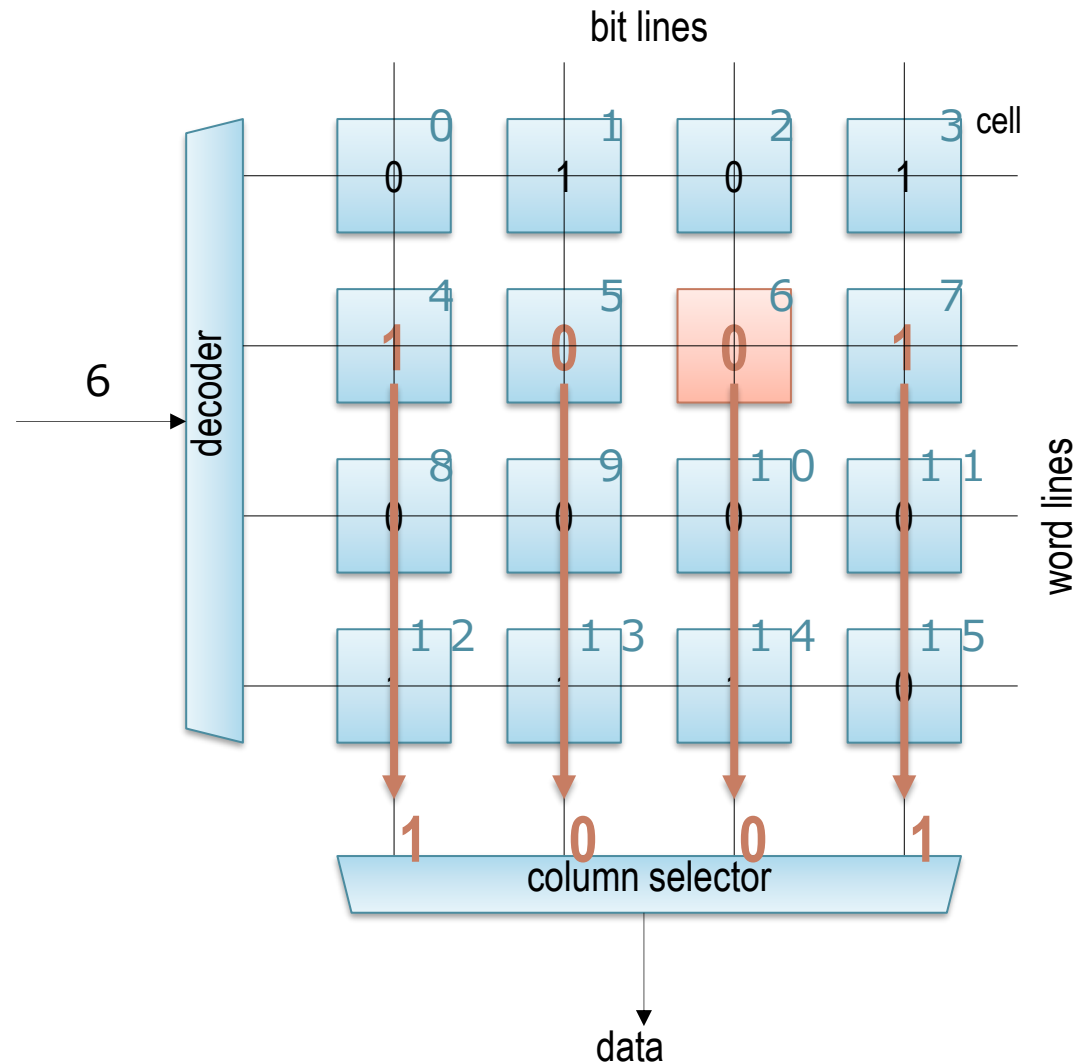
- 数字を対応するワンホット信号に変換する回路
- ワンホット信号：
 - n 本のうち、1つだけが1で他が0の信号
- アドレス上位の2ビットをデコード

メモリの読み出し動作（２） ワードラインのアサート



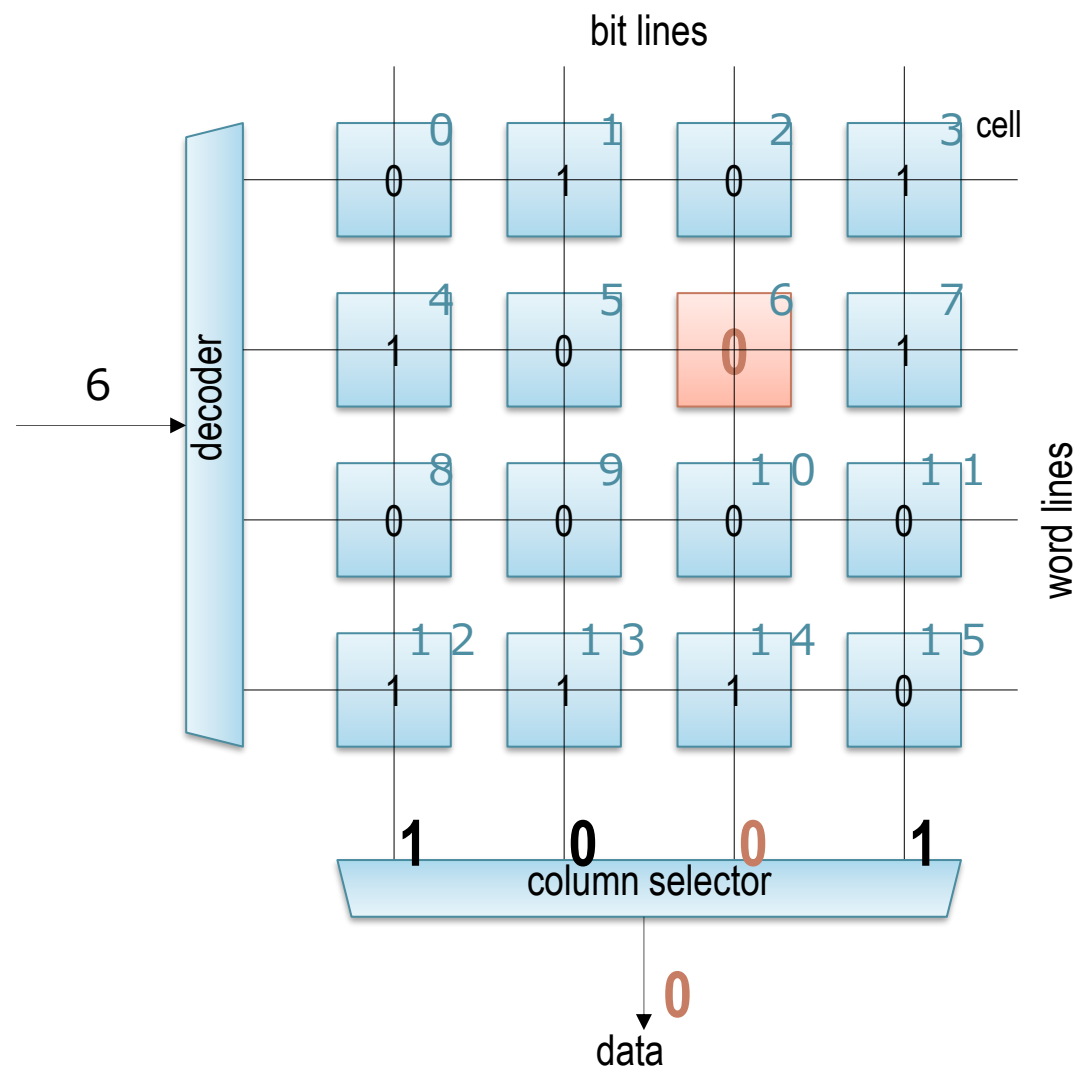
- 読み出す行に対応したワードラインをアサート
- 先ほどのデコード結果を使う

メモリの読み出し動作（３） ワード（１行分のデータ）の読み出し



- ワードラインがアサートされると、そこに接続されたセルがビットラインに自身の中身を流す
- これにより、１行分のデータが読み出される

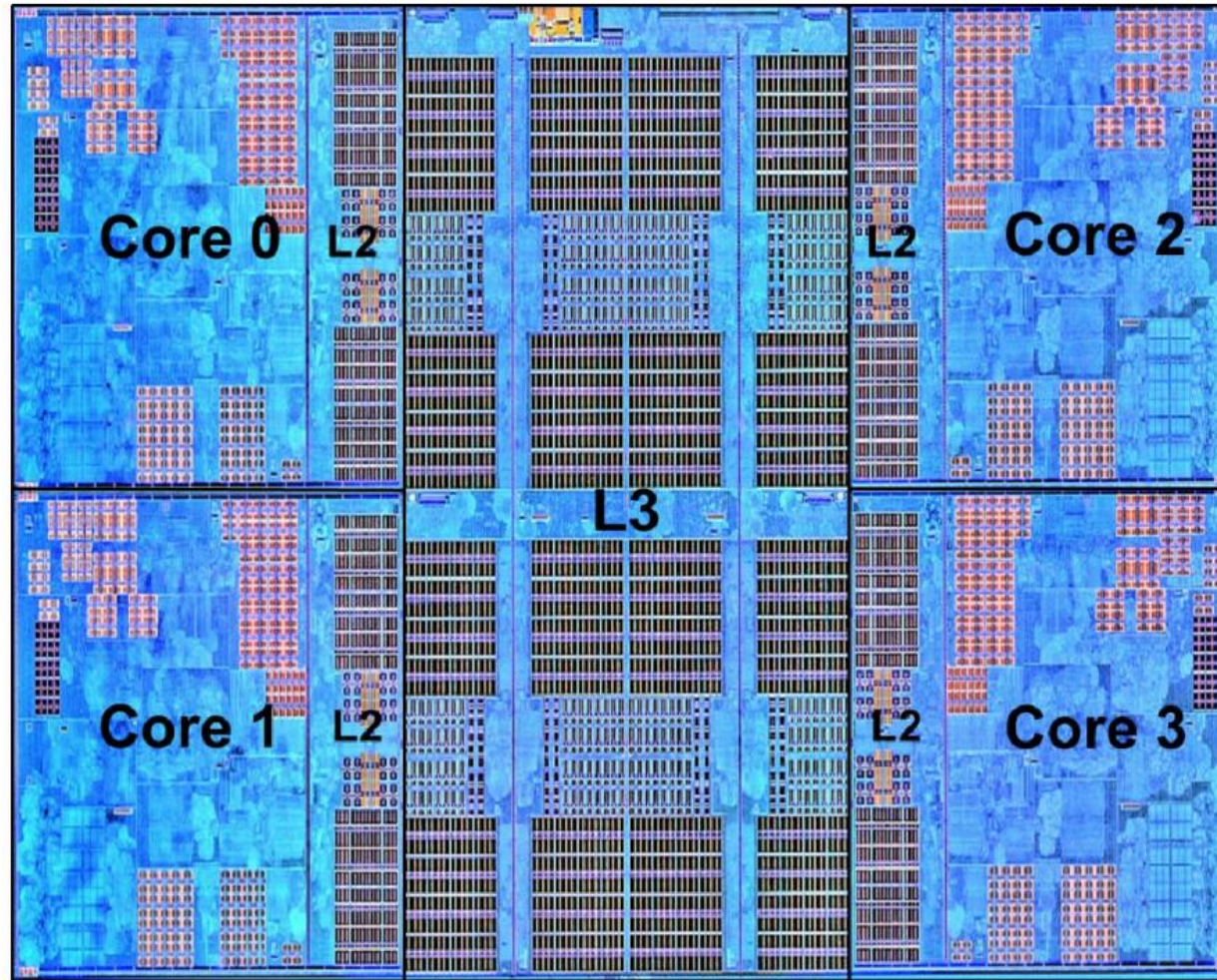
メモリの読み出し動作（４） 列の選択



- 読み出された 1 行分のデータから、最終的に必要な 1 ビットを選択
- どの列を読むか決める
 - 各列は 4 つセルがある
 - アドレスを 4 で割った余りの列が読むべき場所
 - $6 = 0110$ （2 進数）
- カラム・セレクトによりビットを選択する
 - 読み出し幅によってはこの部分がないこともある

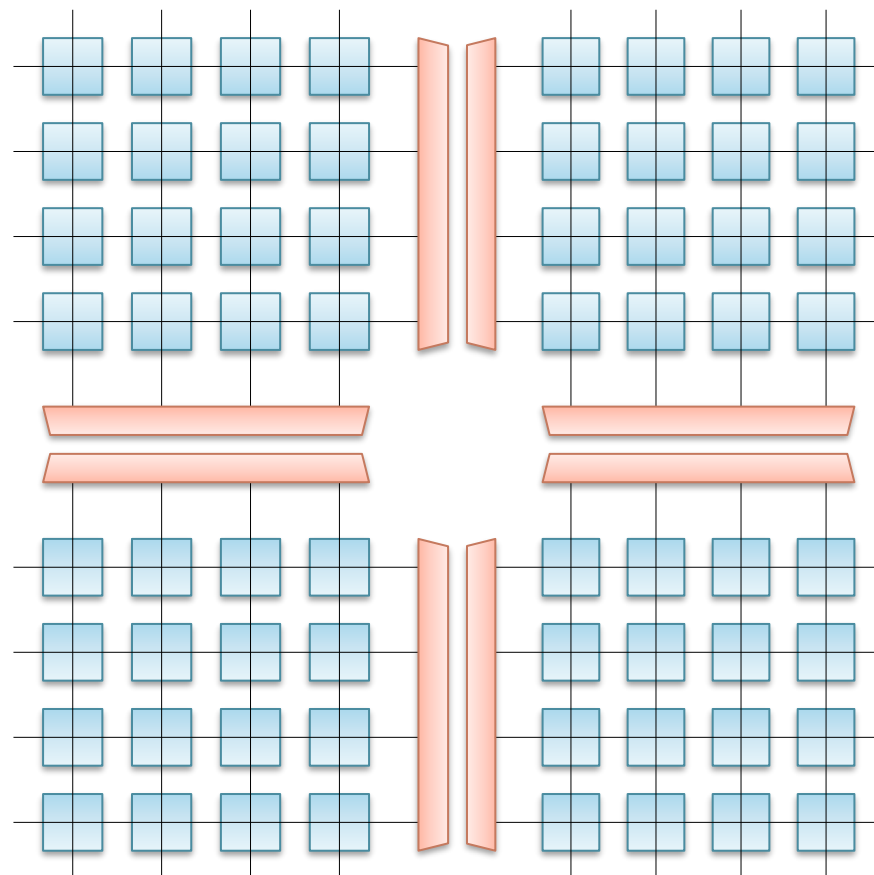
AMD Zen コアの場合

Teja Singh et al., Zen: An Energy-Efficient High-Performance x86 Core より



- 赤や紫の「田」の字の構造の部分が全部メモリ（SRAM）
 - レジスタやキャッシュ，各種テーブルなど

「田」の字の構造



- 「田」の字の構造になっているのは,
 - 縦線がデコーダで左右に対してワードラインをアサート
 - 横線でビットラインのデータを拾う

メモリ

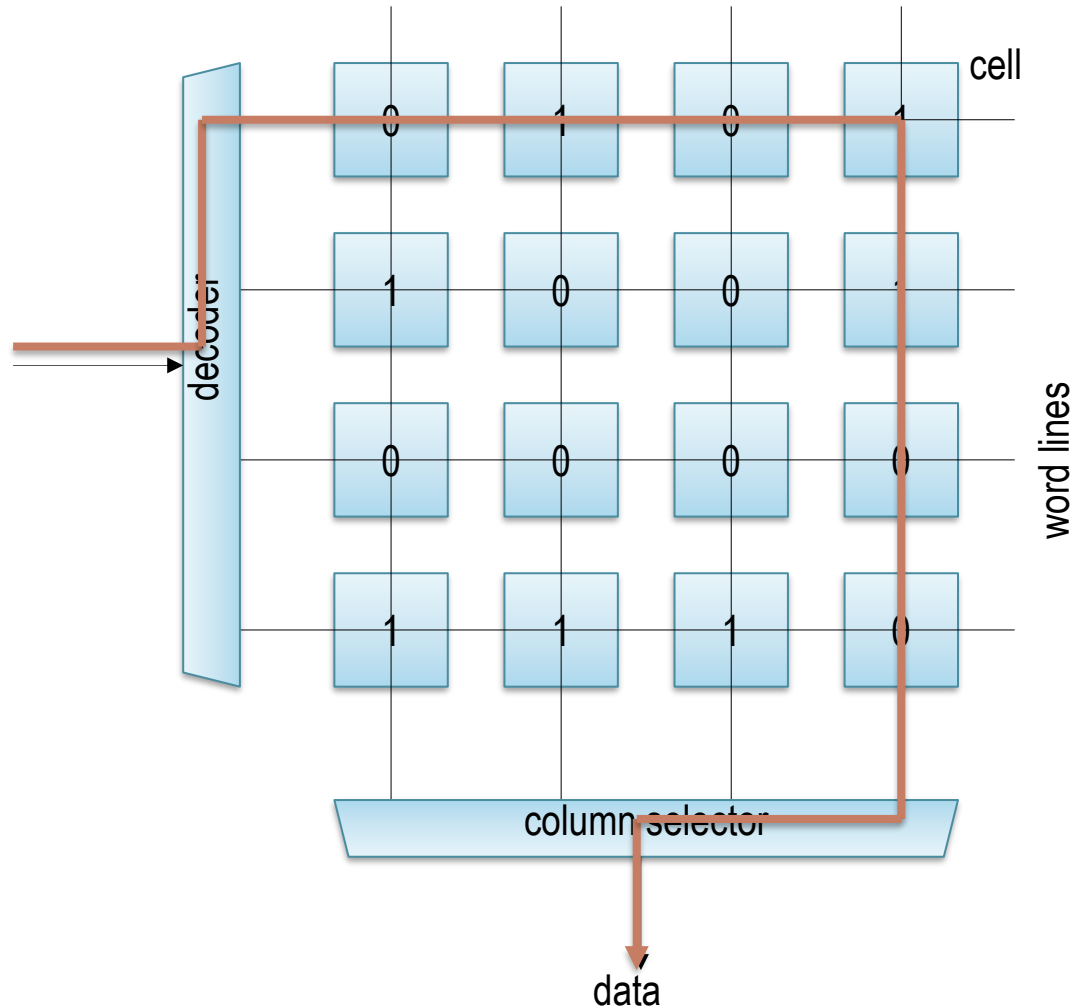
1. メモリの基本

1. 構造
2. 動作
3. **アクセス時間**

2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリを対象にしたアタック

メモリの読み出し時間



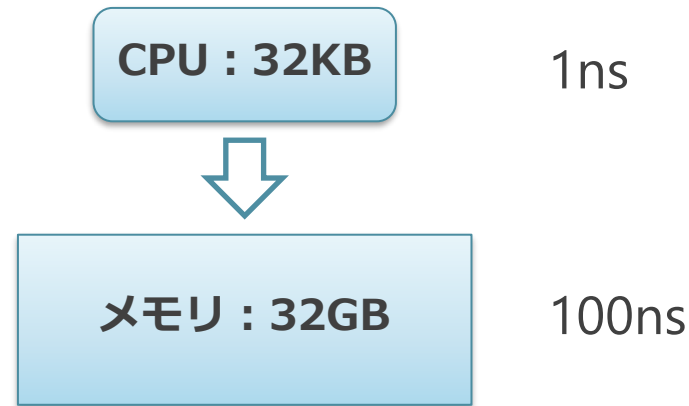
- メモリの読み出し時間：
 - 一番遠い経路を信号が通るときの時間
- 容量（セルの数）が一定の場合，正方形に近くするのが最も経路が短くなる

データをとってくるのに、どのぐらいかかるか？



アクセス時間は容量に直接は比例しない

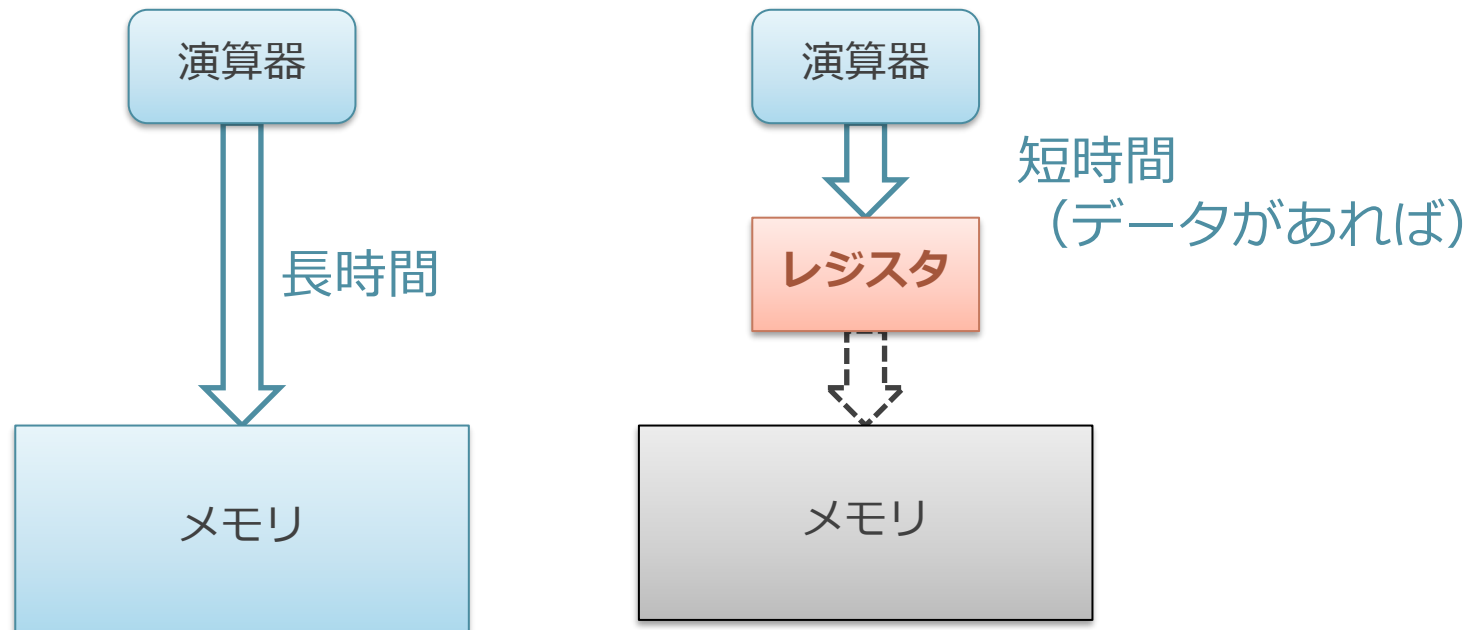
(容量の平方根ぐらいに比例)



- 格子状に並んだセルの外周部の長さがアクセス時間を決めるから
 - メモリ容量 = (外周部の長さ)²
 - アクセス時間は, 容量の 0.5 乗でしか伸びない

なぜレジスタやキャッシュがあるのか？

- 小容量だけど、高速なレジスタやキャッシュを用意
 - 一度利用した値を入れておくことで、2回目からは高速に
- **大容量かつ高速なレジスタややキャッシュは作れない**
 - 記憶容量に応じたアクセス時間が必要だから
 - 最終的には信号が伝わる長さに帰着する



1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリを対象にしたアタック

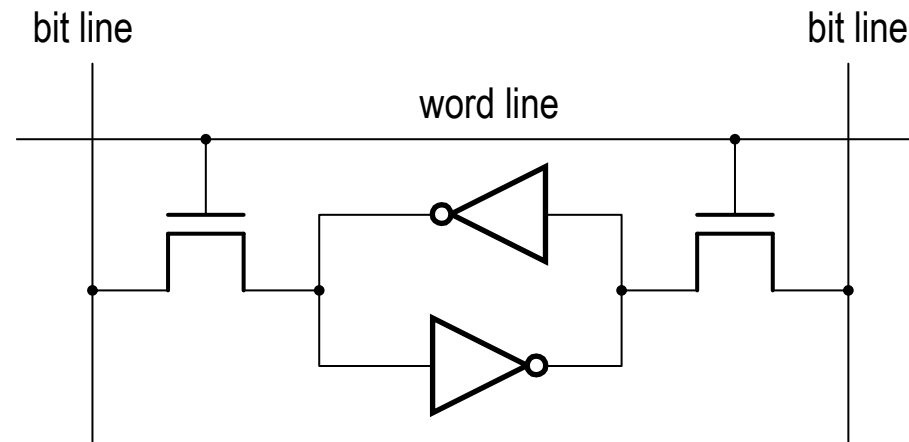
メモリのバリエーション

- メモリは基本的にはみな同じ構造を取る
 - セルの中身が方式ごとに異なる
- SRAM と DRAM を紹介

Static Random Access Memory (SRAM)

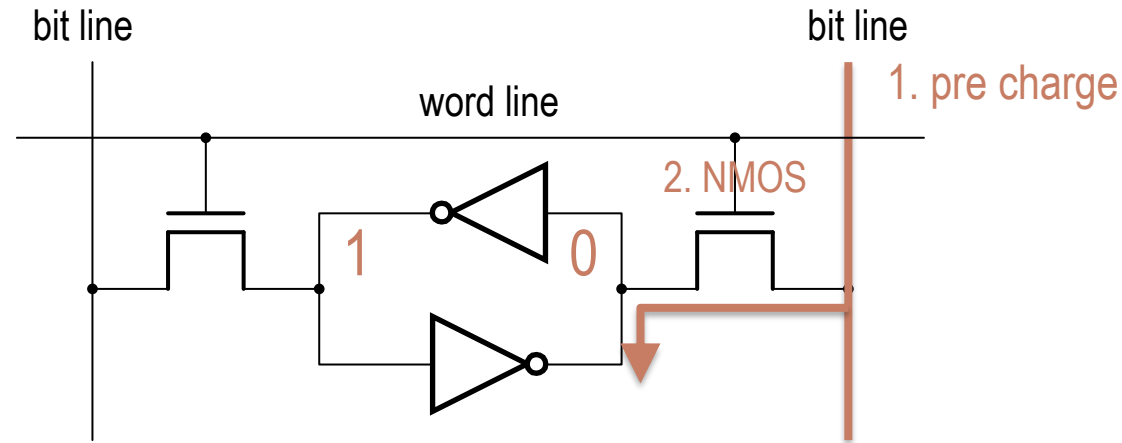
- Static とは :
 - 動作が静的（何もしなければ電流をほとんど消費しない）
 - 電源を入れている限りは内容が消えない
- 一般的にはメモリの中で最も高速
 - CPU 内の論理回路と同じトランジスタを使って作るから
 - レジスタやキャッシュ, 各種テーブルは SRAM で作られている

SRAM のセル (1bit)



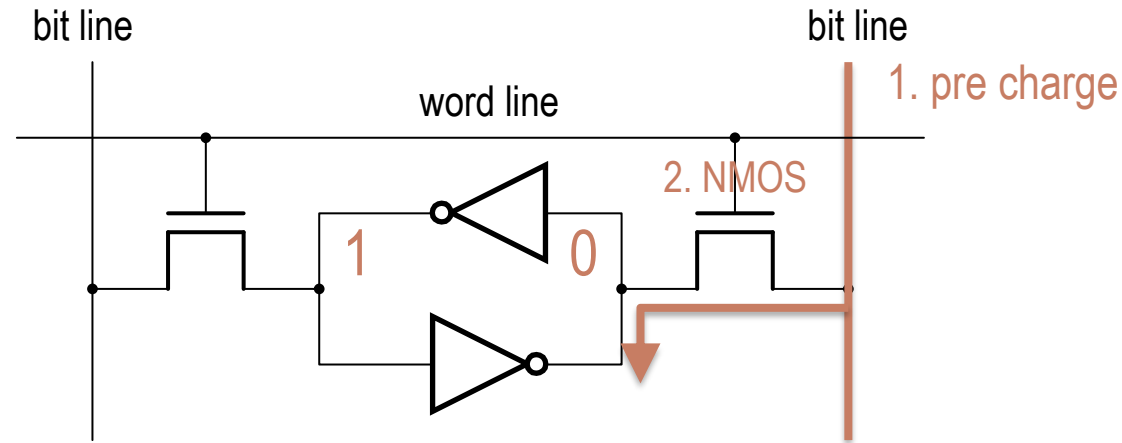
- インバータのループにより, 1 bit を記録
 - ループの左右が [1:0] あるいは [0:1] の 2 つの定常状態を持つ

SRAM の読み出し



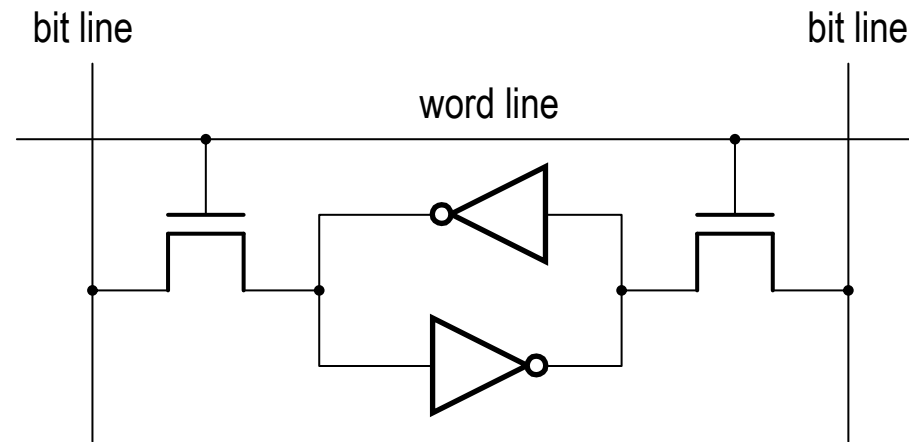
1. ビットラインのプリチャージ
 - ビットラインをあらかじめ高電位（1）にチャージする
2. ワードラインをアサート
 - ループの右にある NMOS が ON に
 - ループとビットラインが接続される
3. ビットラインのディスチャージ
 - ループの右が 0 なら, ビットラインが 0 に
 - ループの右が 1 ならそのまま

なぜプリチャージが必要なのか？



- 単にループとビットラインが接続されるだけではダメ
 - NMOS が高電位をうまく伝えられないから
 - もしループの右側が 1 で NMOS が ON になっても、ビットラインを 1 に引き上げることはできない
 - PMOS を追加すればできるが、なるべく小さく作りたいから嫌だ
- NMOS を介してビットラインの電位を下げるだけができる
 - = あらかじめ電位を上げておいて、下がったかどうかで判定

SRAM の書き込み

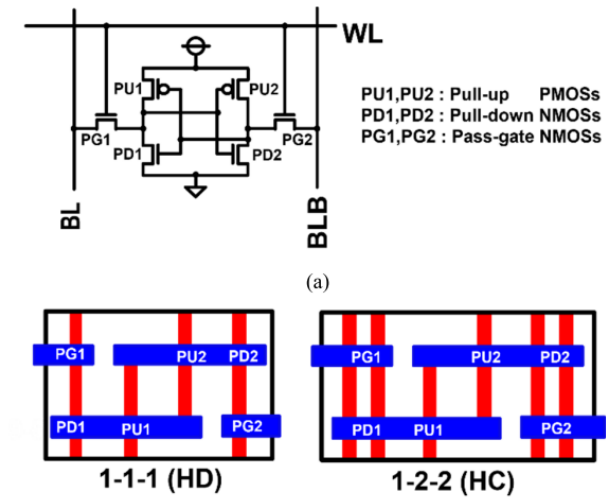
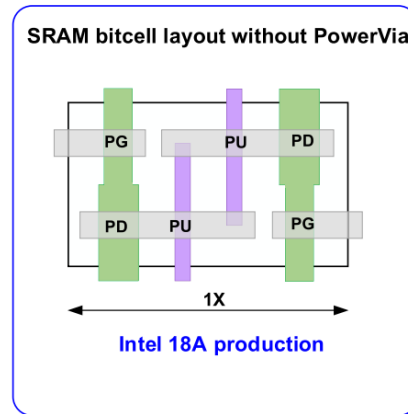
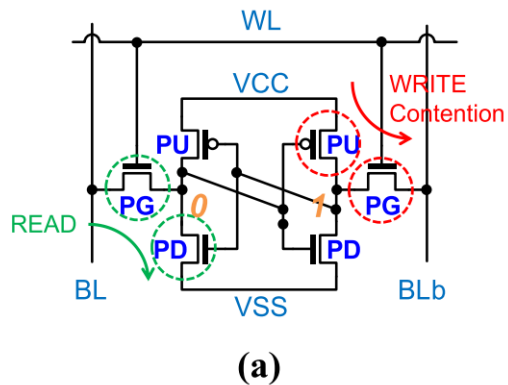


■ 書き込み手順

1. ループ左右のどちらか 0 にしたい方のビットラインの電位を下げる
2. ワードラインをアサートして NMOS を ON に
3. インバータの状態を強制的にビットライン側から低電位にする

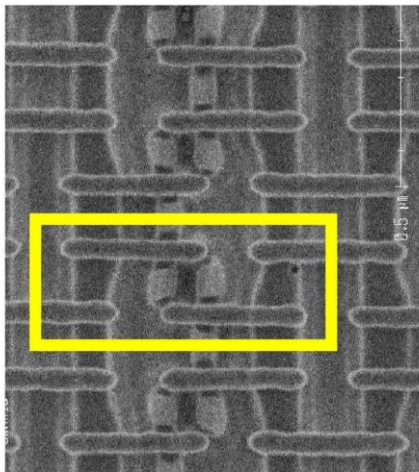
■ NMOS が低電位しか通せないなので、書き込みには 2 本いる

チップ上のSRAMセル



WANG et al.: 0.021- μm^2 HIGH-DENSITY SRAM IN INTEL 18A RibbonFET TECHNOLOGY, 2026 より

Aoyagi et al.: A 3-nm FinFET 27.6-Mbit/mm² Single-Port 6T SRAM Enabling 0.48–1.2 V Wide Operating Range With Far-End Pre-Charge and Weak-Bit Tracking, 2024 より



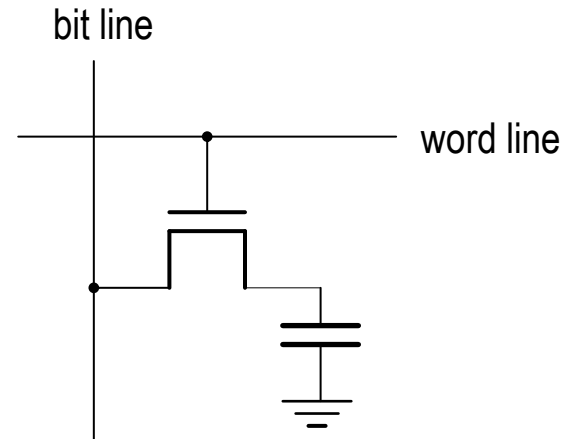
Cheng et al.: A Highly Scaled, High Performance 45nm Bulk Logic CMOS Technology with 0.242 μm^2 SRAM Cell, 2007 より

- 2000年代にこのトランジスタ配置が発明されて以来、ずっと使われている
 - 高密度に配置できることに加え、
 - すべてのトランジスタが同じ向きになっているために微細化時のリソグラフィに優しいらしい
- SRAM の電子顕微鏡写真でもよくわかる
 - 上下左右にミラーされていることに注意

Dynamic Random Access Memory (DRAM)

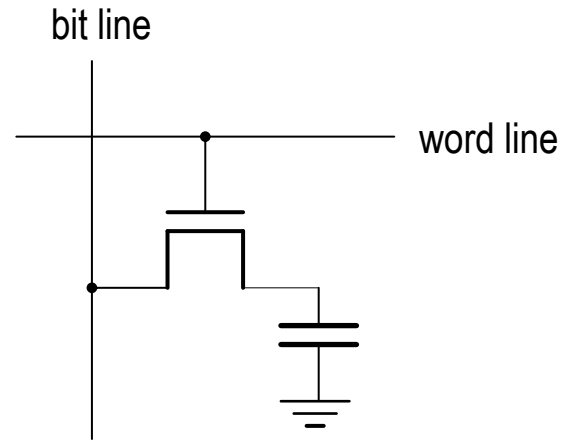
- 速度は遅いが、セルを小さく作れるので容量が稼げる
 - メイン・メモリは DRAM で出来ているのが普通

DRAM のセル



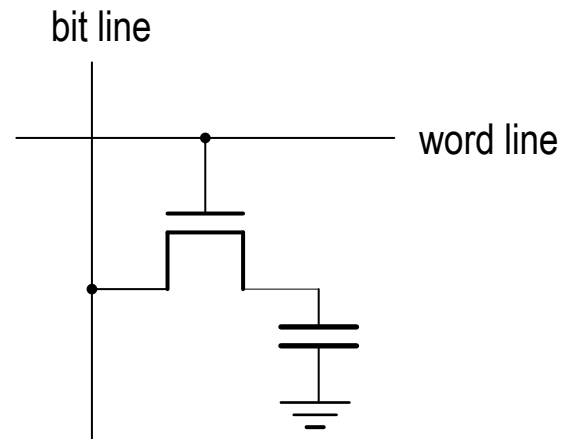
- セルをコンデンサによって構成
 - 電荷がたまっていれば 1, そうでなければ 0
 - 時間が経つと自然に電荷が抜けてデータが消える
- 電荷を維持するため, 定期的にはリフレッシュと呼ばれる操作を行う
 - 具体的には, 一回読んで同じデータを書き戻す

DRAM の読み出し



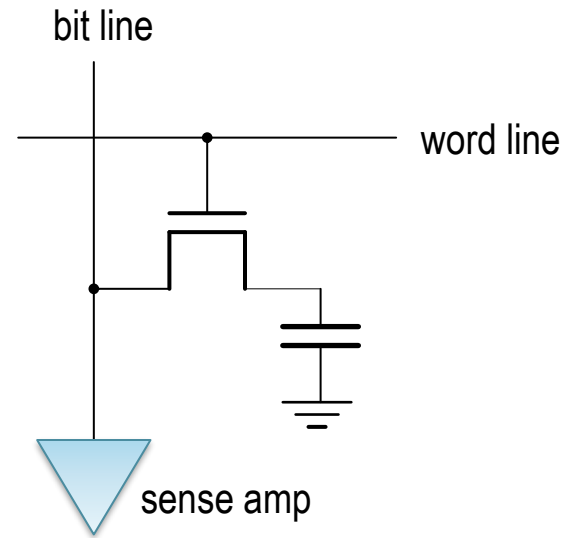
- 読み出しは, 基本的には SRAM と同じ
 - ビットラインをプリチャージ
 - ワードラインをアサート
 - セルの状態に応じてディスチャージが起きる

DRAM の書き込み



- 書き込みは, NMOS でも 0.5 までは上げられるのでそれで行う (多分)
 - SRAM ではインバータをひっくり返す必要があるので, 0.5 ではダメだった
- DRAM では 低電位 (0) or 中間電位 (0.5) を記録する
 - 読み出し時のプリチャージの電圧を 0.5 にして, 下がったかどうかで判定

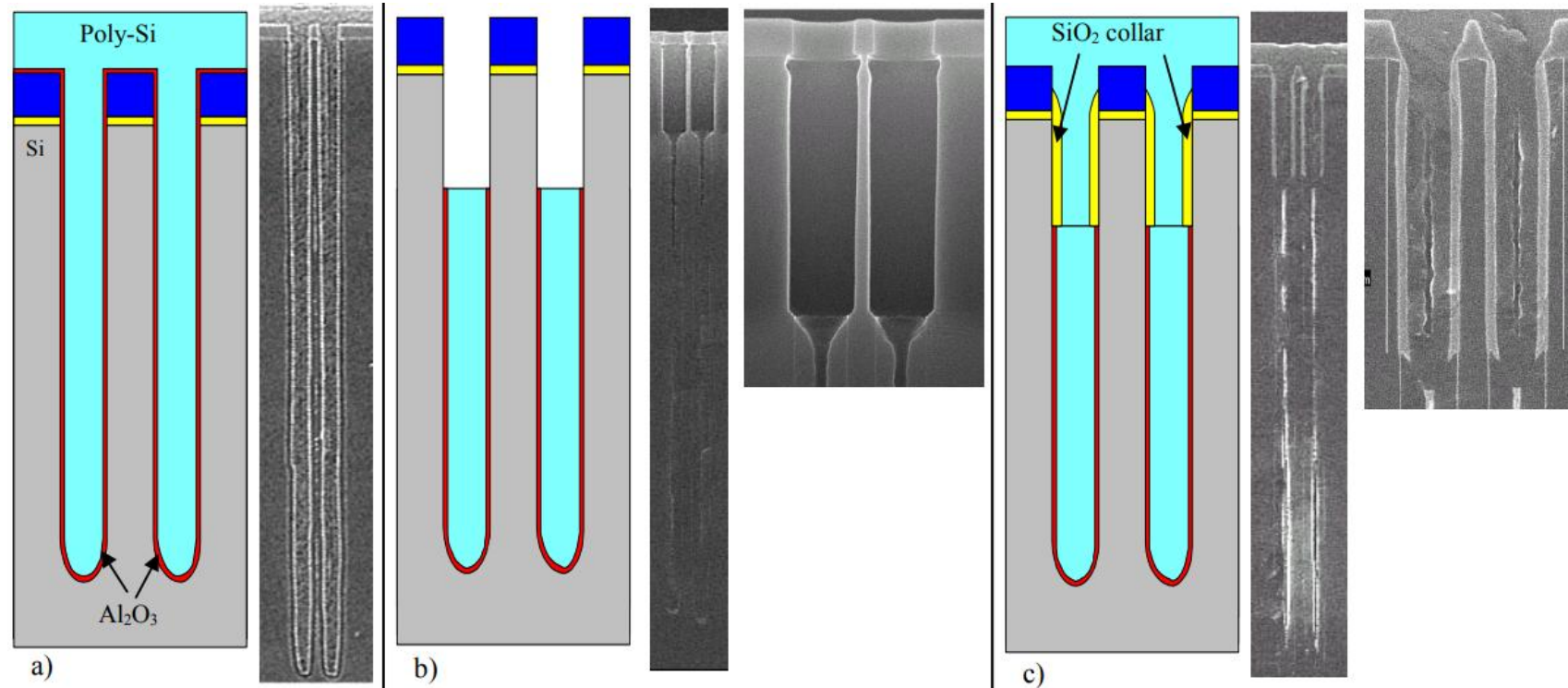
SRAM の場合との違い



- 読み出し時に内容が破壊される
 - ビットラインとコンデンサが接続されると、電荷が流れ出す
- コンデンサがビットラインをディスチャージする速度は遅い
 - センスアンプと呼ばれる微小な電圧の変化を増幅する回路を使う

DRAM のコンデンサの作り方

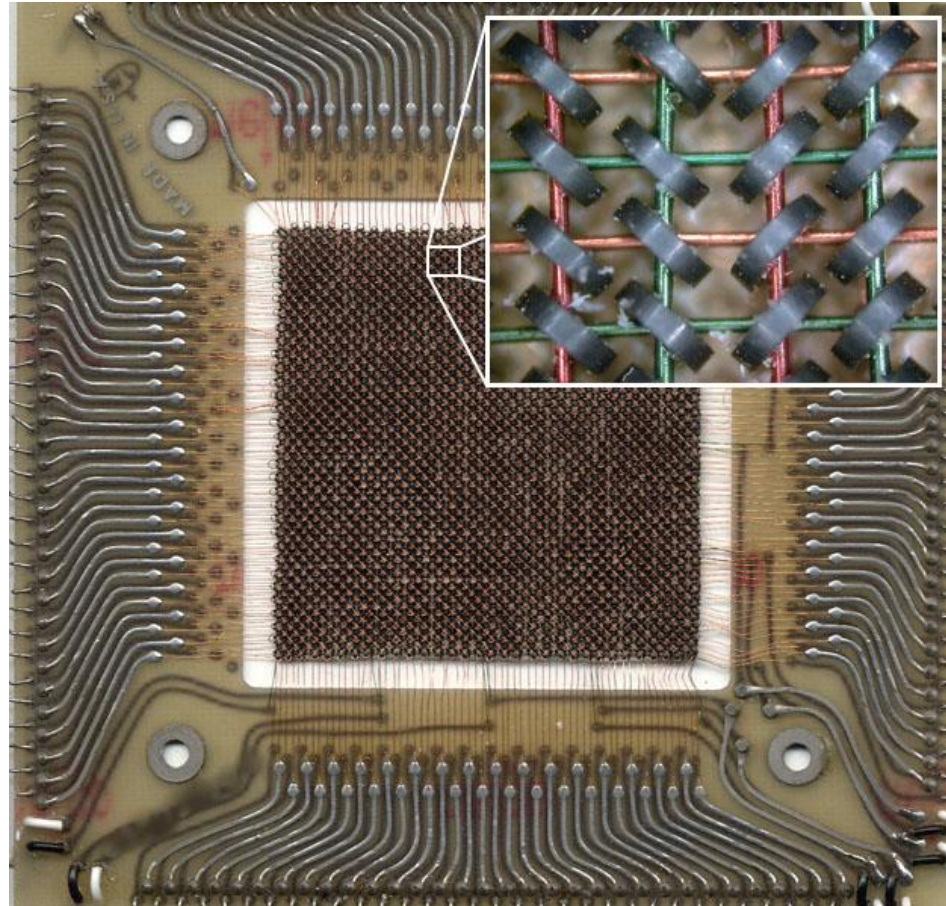
H. Seidl et al, A Fully Integrated Al₂O₃ Trench Capacitor DRAM for Sub-100nm Technology より



- チップの表から裏に向けて深い穴を掘って表面積を稼ぐ
 - コンデンサの容量は表面積に比例
 - トレンチ（塹壕の意味）と呼ばれる
 - 特殊な工程が必要なので通常のトランジスタと混ぜて作るのが難しい

余談：磁気コアメモリ

<https://ja.wikipedia.org/wiki/%E7%A3%81%E6%B0%97%E3%82%B3%E3%82%A2%E3%83%A1%E3%83%A2%E3%83%AA> より



- 1950年代から使用されていた
- いわゆる「コアダンブ」のコアとは元々この磁気コアのこと

メモリ

1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

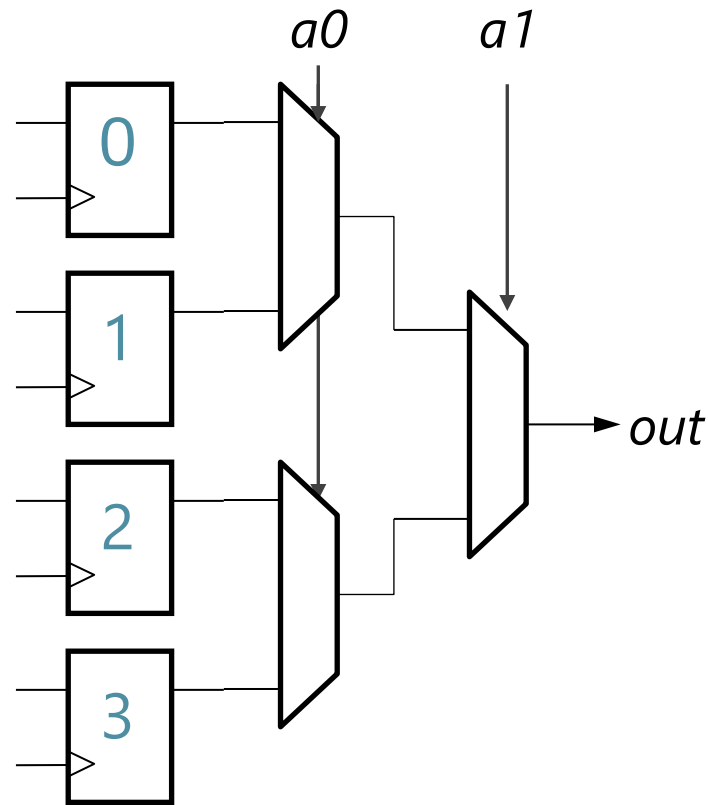
2. メモリの詳細

1. SRAM と DRAM
2. **なぜメモリが存在するのか？**
3. マルチポート・メモリ
4. メモリを対象にしたアタック

メモリの存在する理由

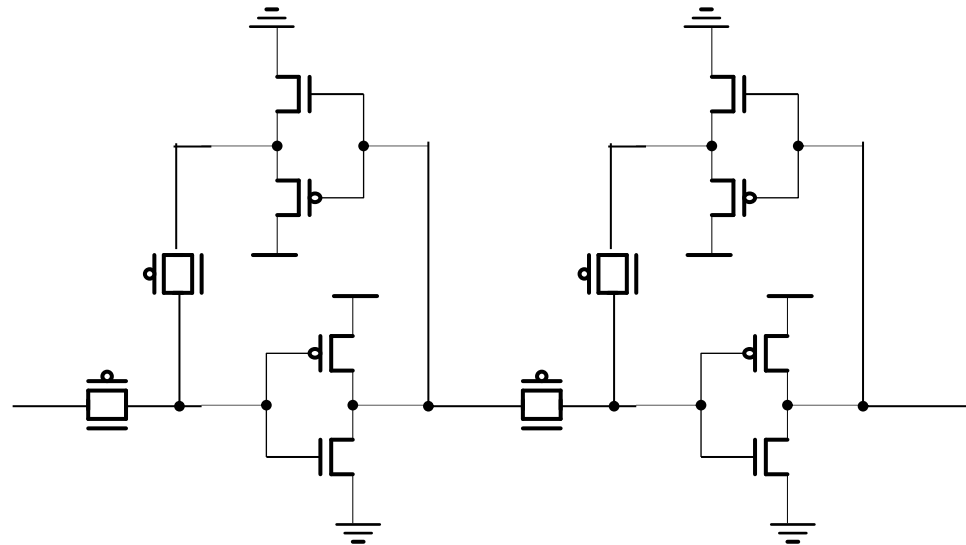
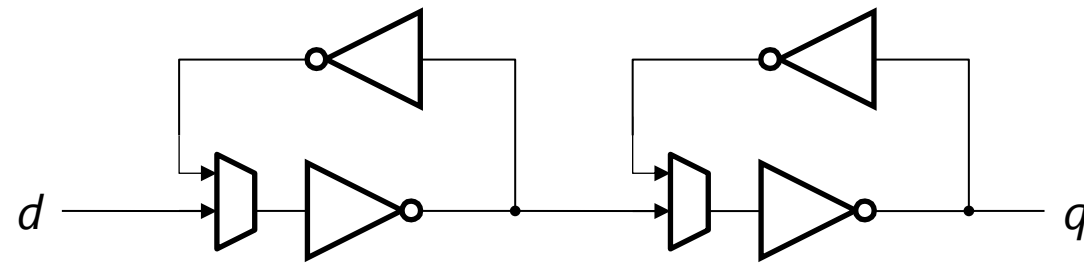
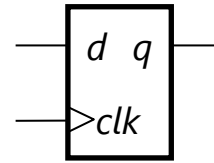
- D-FF とマルチプレクサがあれば、等価な機能は実現できる
 - 実際には、メモリ専用の回路が用意される
- メモリ専用の回路が用意される理由：
 - D-FF とマルチプレクサよりも、高密度に実装したいから
 - 同じチップ面積で、より多くの情報を記憶したい

D-FF とマルチプレクサによる 4bit の RAM

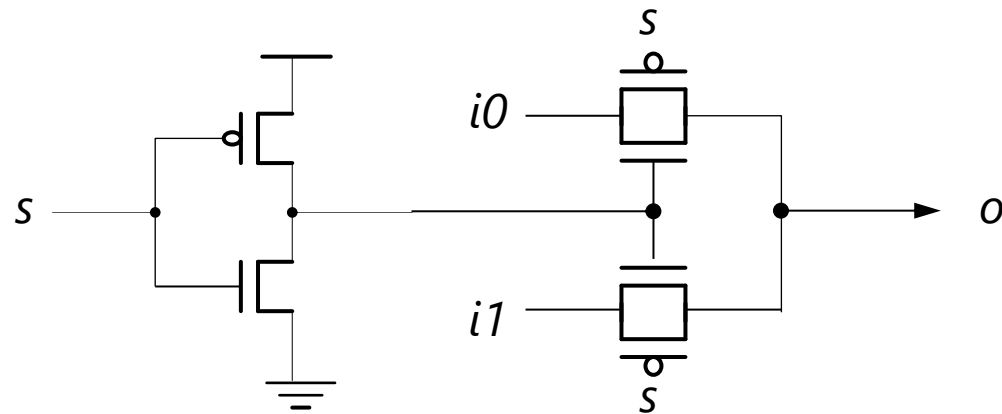
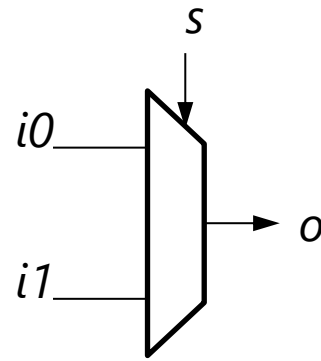


- 4 エントリの LUT を D-FF で構成してみる
 - 中身を憶える 4つの D-FF
 - 場所を指定して選択する2段のマルチプレクサ

D-FF : トランジスタ 16個

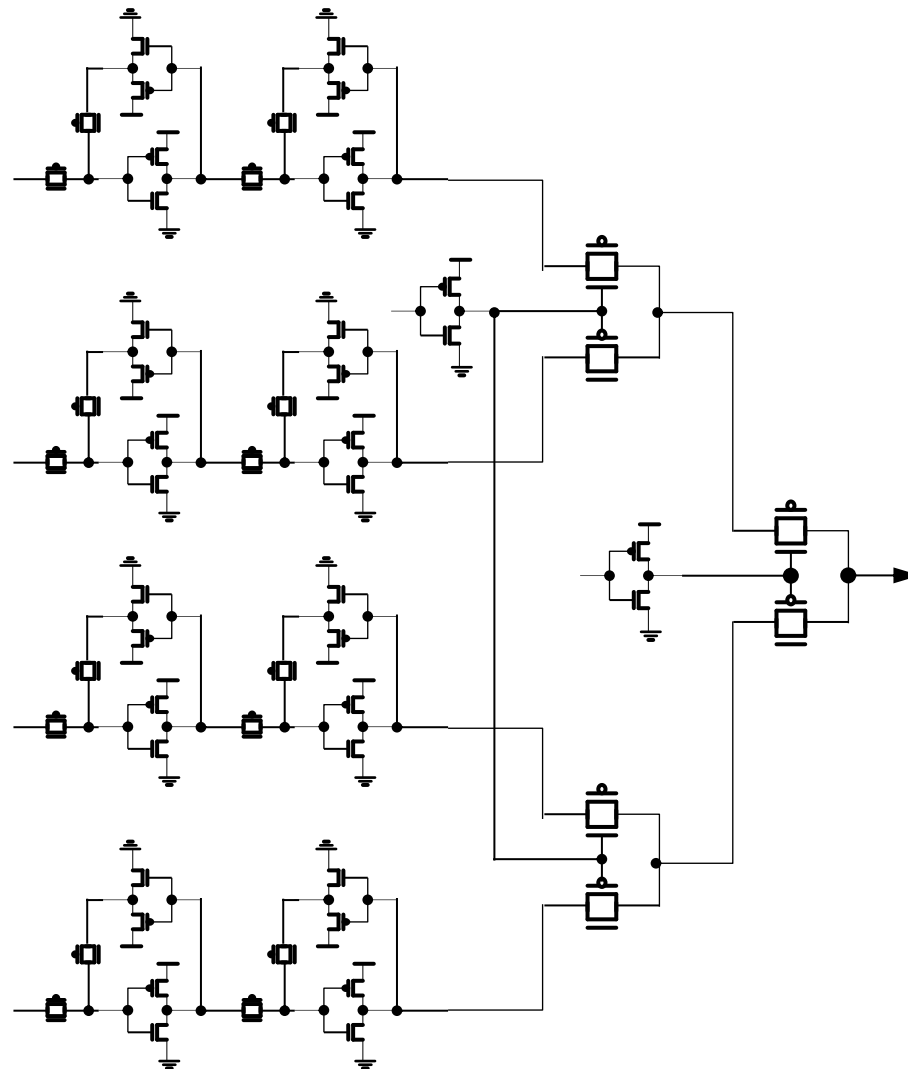


マルチプレクサ：トランジスタ 6個



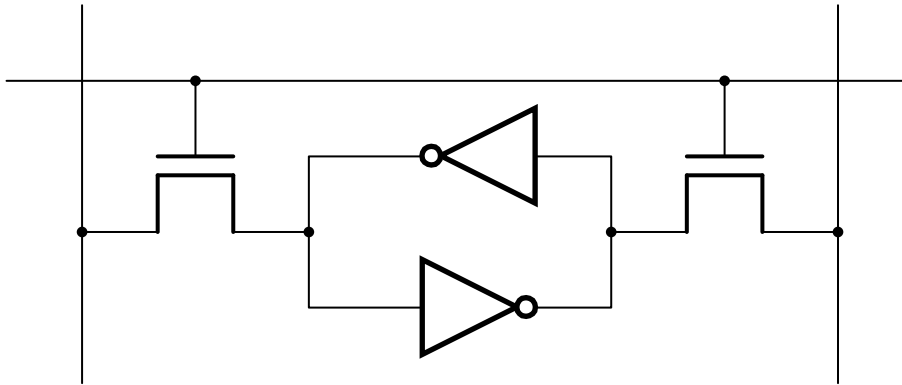
4bit メモリに必要なトランジスタ数 : 80

$$16 \times 4 + 6 \times 3 = 82$$

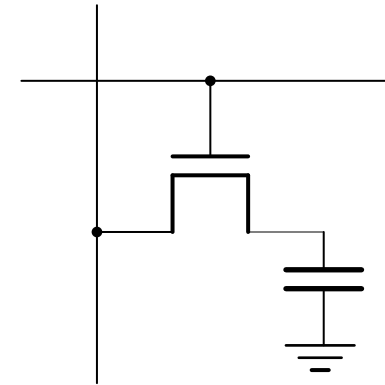


SRAM と DRAM のセル (1bit)

SRAM : $2 + 2 \times 2 = 6$



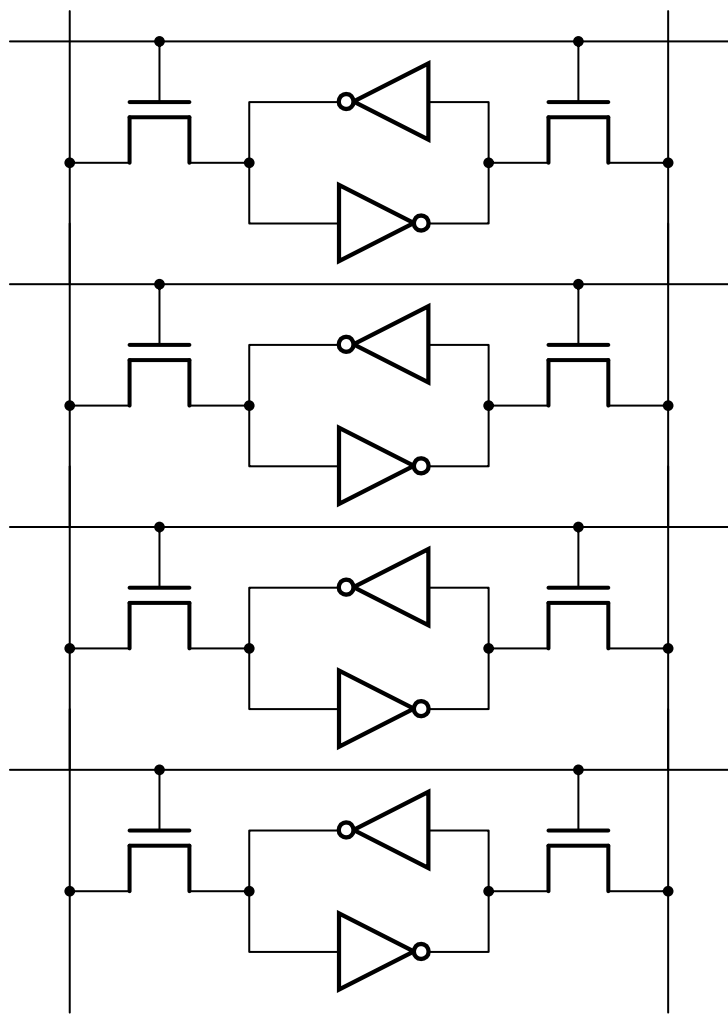
DRAM : 1 + コンデンサ 1



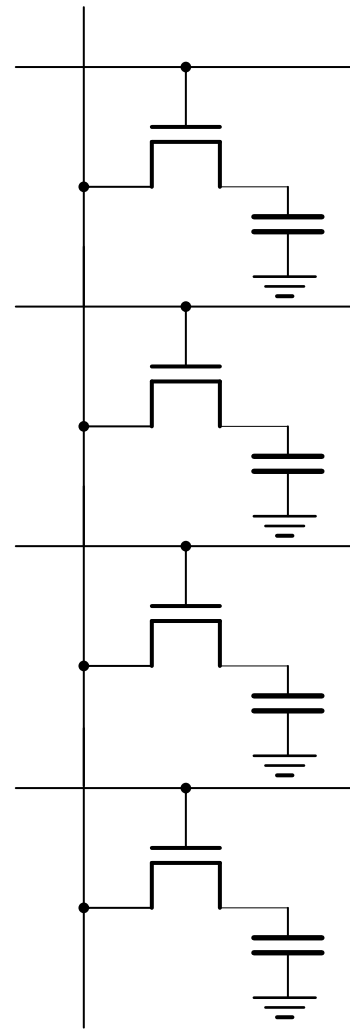
- D-FF よりも圧倒的に単純

SRAM と DRAM (4bit)

SRAM : $6 \times 4 = 24$



DRAM : $4 + \text{コンデンサ } 4$



メモリ

1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリを対象にしたアタック

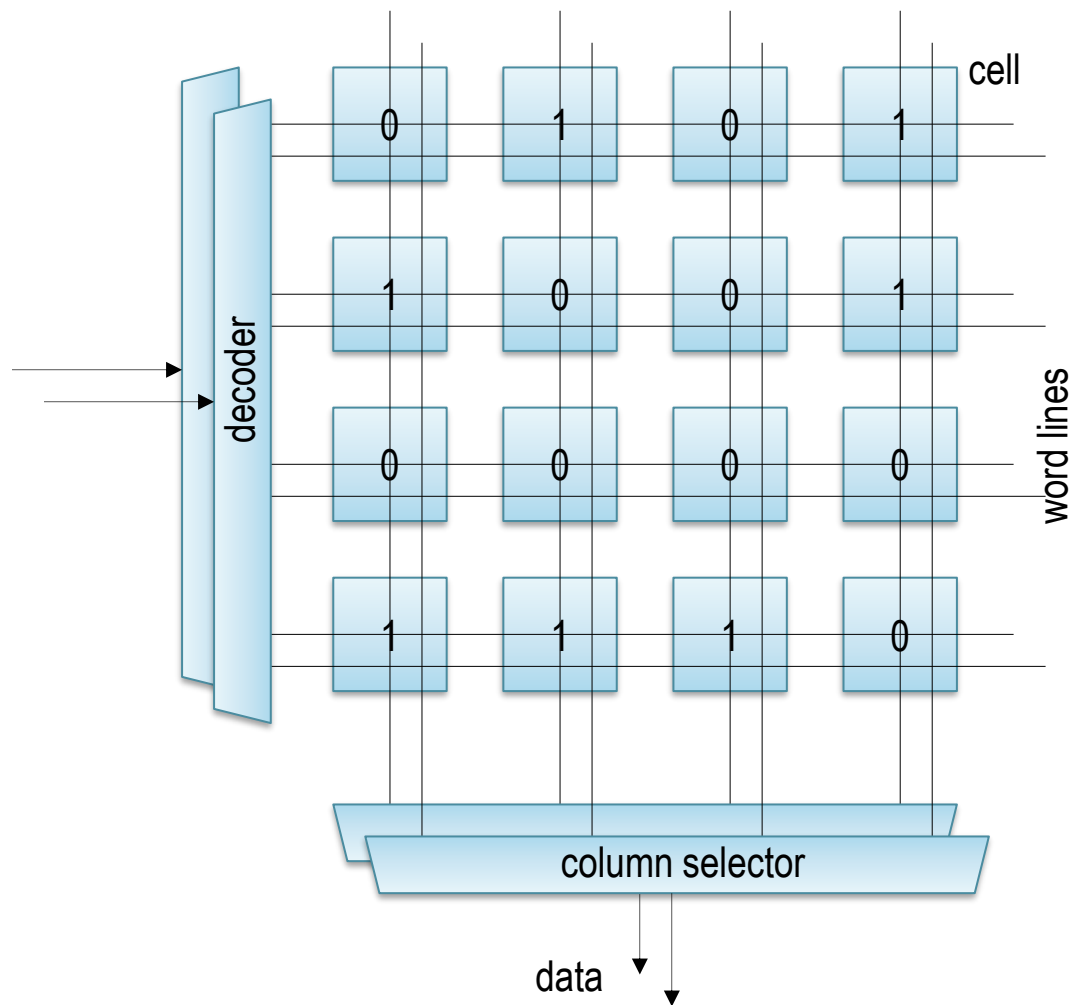
マルチポート・メモリ

- ランダムな複数の箇所を同時にアクセスできるメモリ
 - レジスタ・ファイルなどを作るときに必要
 - 2つのソースの読み出しと, ディスティネーションの書き込み
 - 同時にアクセスできる数 = ポート数
- CPU に特有に良く必要になる
 - 一般的な回路ではあまり出てこない
- ポート数の2乗に比例して面積が大きくなる
 - 同時にアクセスできる数の2乗に比例する

マルチポート・メモリの回路面積

- ポート数の2乗に比例して面積が大きくなる
 - 同時にアクセスできる数の2乗に比例する
- レジスタ・ファイルなどは、同一記憶容量あたりの回路面積が大きい

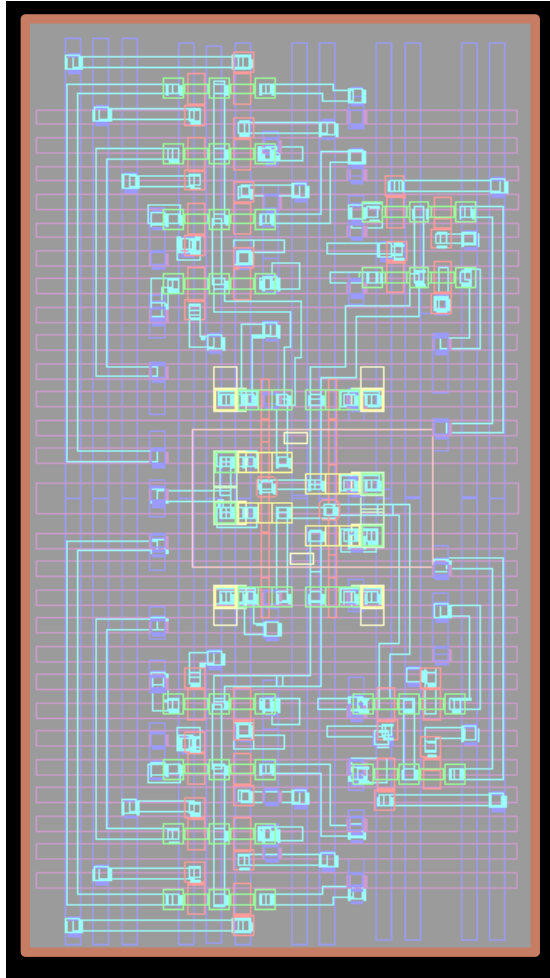
ポート数の2乗に比例して面積が大きくなる



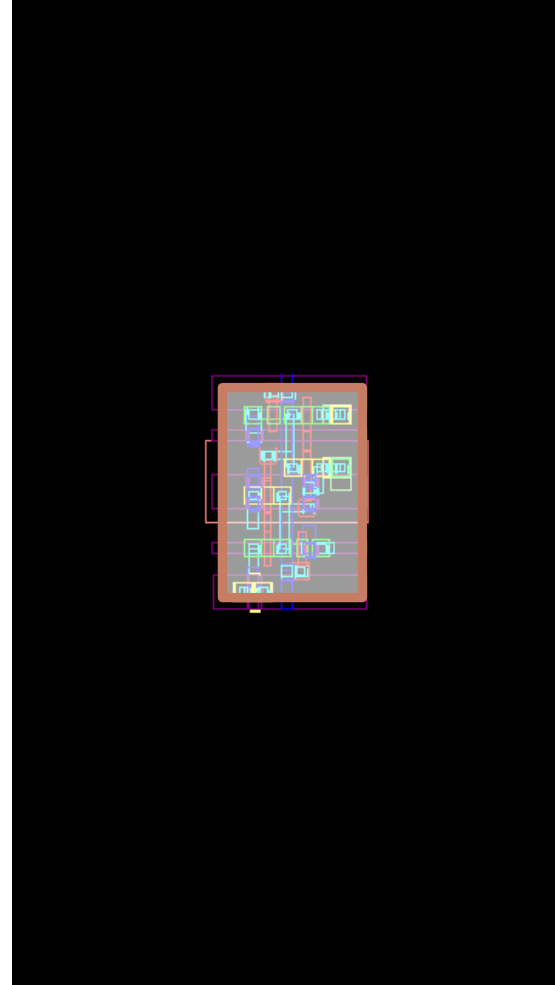
- 複数の異なる行をアクセス
 - ワードラインが複数必要
 - ビットラインも複数必要
- 縦と横がそれぞれポート数分増える
 - ポート数が増えると配線の面積が支配的に

SRAM (1bit) のレイアウト

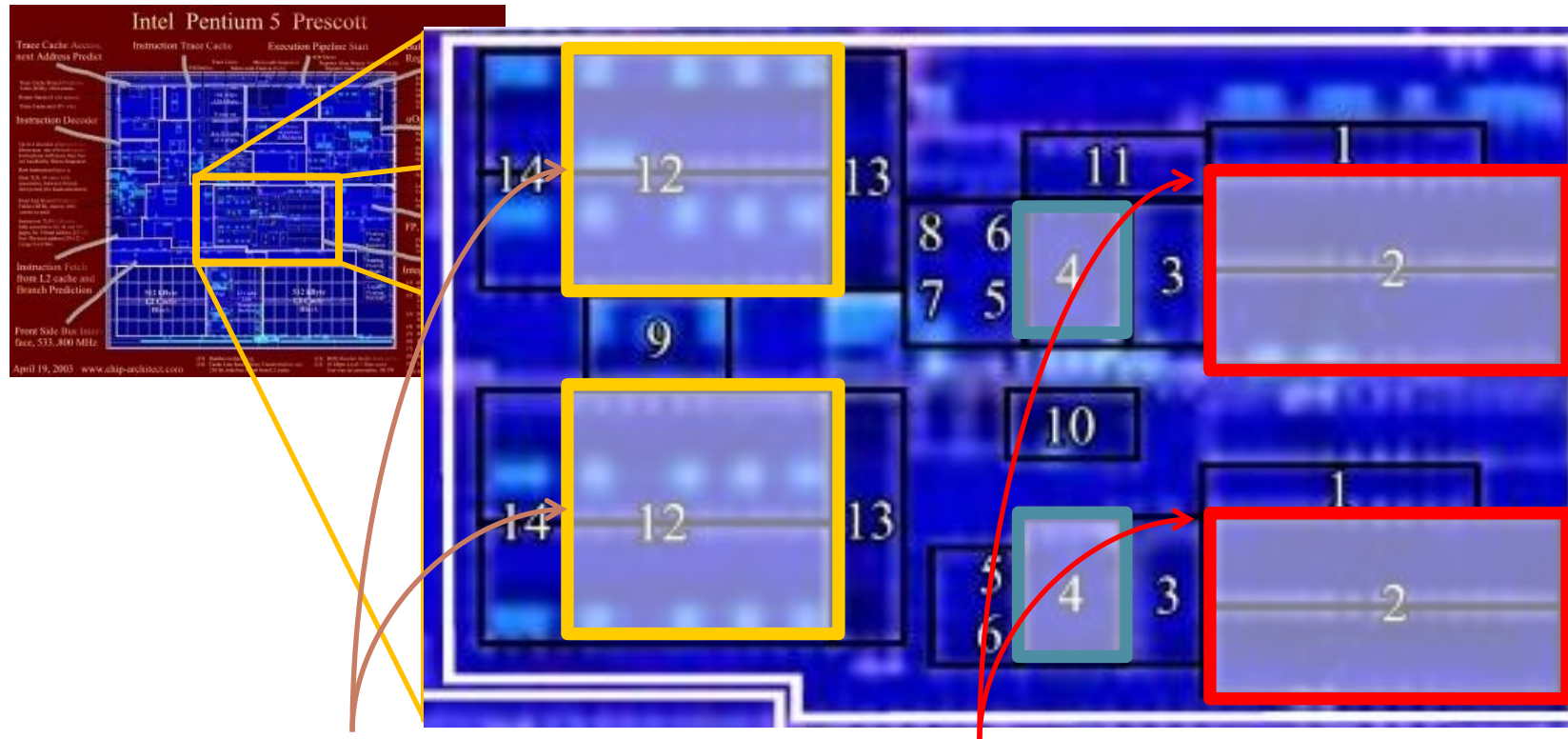
12 ports



2 ports



Intel Pentium 4 の整数演算ユニット



L1 Data cache

16KB 1 read/1 write

(from <http://www.chip-architect.com>)

Register file

64 bits 144 entries 6 read/3 write

(double pumped)

マルチポート・メモリと CPU の回路規模

- CPU の性能を上げようとするすると、ポート数が増える事が多い
 - 例：2 命令を同時に処理できるようにする
 - レジスタ・ファイルの読み出しと書き込みが倍に
 - ポート数が倍 → 回路面積 4 倍

マルチポート・メモリと CPU の回路規模

- CPU はそういったメモリのかたまり
 - 演算器よりもメモリの方がずっと大きい
 - 投機実行や並列実行をするために様々なテーブルが導入される
 - それらは同時アクセス可能な数に対して、急速に大きくなる
- 同時実行数を増やすことが難しい
 - 完全に独立したメモリなら問題ない
 - 1つの大きなコアを作るより、お互いが独立しているマルチコア化の方が面積効率は良い

メモリ

1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

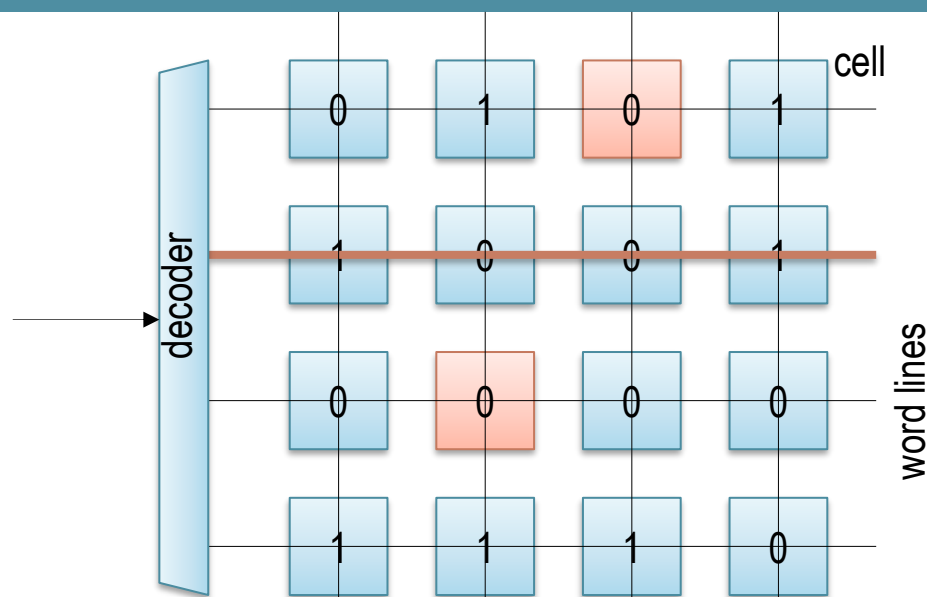
2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリを対象にしたアタック

メモリを対象にしたアタック

- メモリの仕組みがわかると、原理がわかる
 1. Row Hammer
 2. Cold Boot Attack

Row Hammer



- 特定の行を過剰に集中してアクセスすると隣接行のデータが化ける
 - 通常よりも DRAM セルの放電が早まるため
 - 意図的にデータを化けさせて様々なアタックを行う
 - 特権昇格とかは実証コードがでている・・・らしい
- Yoongu Kim et al., Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors など

Row Hammer の応用 : リバースエンジニアリング

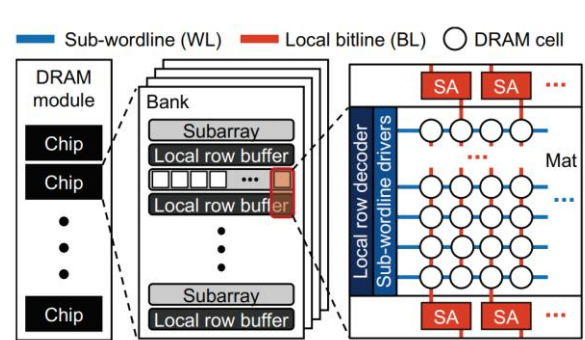


Figure 1. Conventional DRAM organization.

Table III
THE STRUCTURE OF SUBARRAYS AND ROWS WE IDENTIFIED.

DRAM type	Vendor	Chip type	Year	Subarray composition	Edge subarray interval	Coupled-row distance
DDR4	Mfr. A	×4	2016	11 × 640-row	per	64K rows
			2017	2 × 576-row (per 8192-row)	16K rows	
			2018	4 × 832-row	per	N/A
		×8	2021	1 × 768-row (per 4096-row)	32K rows	
			2017	11 × 640-row	per	N/A
			2019	2 × 576-row (per 8192-row)	16K rows	
	Mfr. B	×4	2018	4 × 832-row	per	N/A
			2019	1 × 768-row (per 4096-row)	32K rows	
		×8	2019	4 × 832-row	per	64K rows
			2018	1 × 768-row (per 4096-row)	32K rows	
			2018	4 × 832-row	per	N/A
		×4	2021	2 × 688-row	per	N/A
HBM2	Mfr. A	4-Hi	2018	1 × 672-row (per 2048-row)	32K rows	
			2016	1 × 688-row	per	N/A
			2019	2 × 680-row (per 2048-row)	4K rows	
		×8	2019	2 × 688-row	per	N/A
			2019	1 × 672-row (per 2048-row)	32K rows	
			2019	1 × 672-row (per 2048-row)	32K rows	

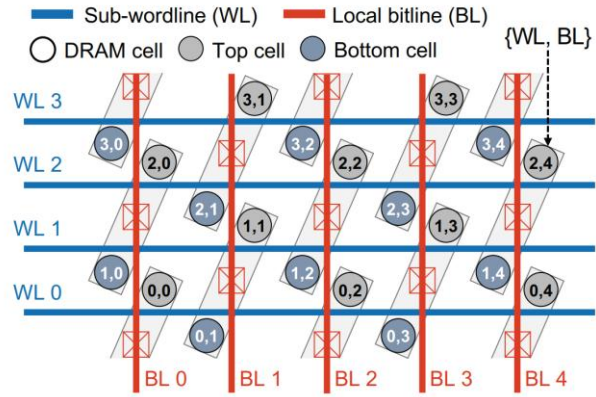


Figure 11. 6F² DRAM cell layout of 4x4 cell array. Each cell represents its position through indices of {WL, BL}.

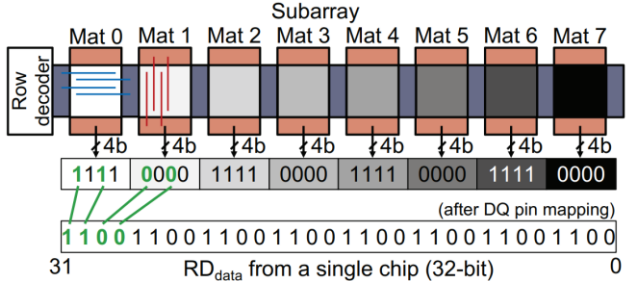


Figure 7. The specific manner in which 32-bit RD_data is physically stored in a Mfr. A's DDR4 ×4 chip.

- アドレスと物理配置の対応はよくわからない
 - DRAM チップ内はかなり複雑な階層構造を持つ
 - アドレスはかなりインタリーブされている
- Row Hammer を行った際に、このデータが壊れるかを観測することで対応を明らかにできる
 - 2 の累乗以外のサイズのアレイがあるなど色々興味深い

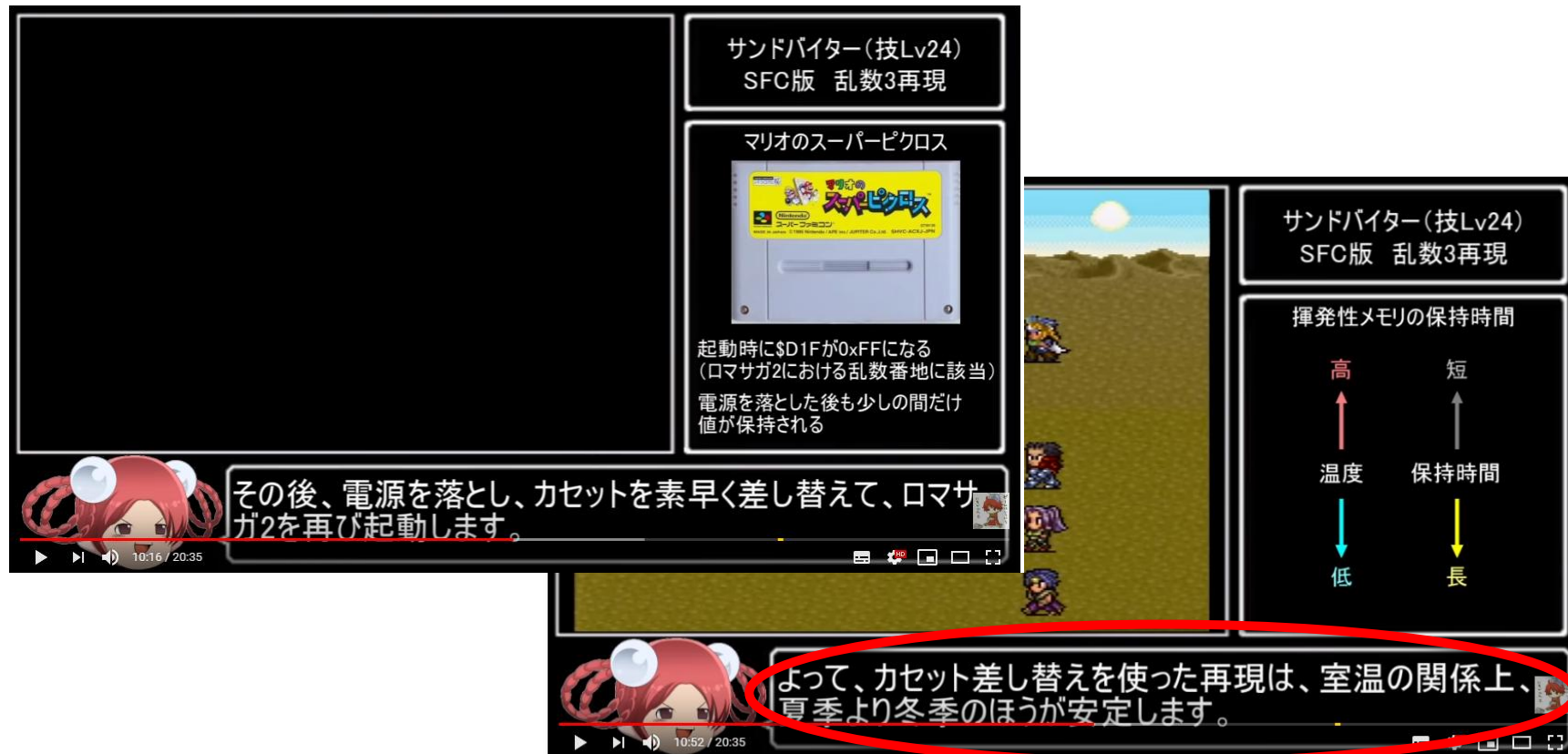
Nam et al., DRAMScope: Uncovering DRAM Microarchitecture and Characteristics by Issuing Memory Commands. 2024 より

Cold Boot Attack

- 稼働中のマシンの DRAM を物理的に引っこ抜いて、他のマシンにさして読み取る
 - DRAM セルの電荷の放電は、温度が低いほどゆっくりになる
 - メモリを極低温に冷やすことで、攻撃成功率が飛躍的に上がる
 - -50 度まで冷やすと、1時間ぐらい持つ・・・らしい
- J. A. Halderman et al., Lest we remember: Cold-boot attacks on encryption keys など

Cold Boot Attack の応用例

- 【ロマサガ2】 ほぼ全ての技を100%閃く方法 (おやつの人, 2019)
 - <https://www.youtube.com/watch?v=gWfcZnAydQ4>



ロマサガ 2 の技のひらめき判定（イメージ）

このゲームでは戦闘中に条件に応じて一定の確率で技をひらめく

```
■ // 乱数値で初期化されたテーブル
uint8_t RAND_TABLE[256] = ...;
...
// 乱数テーブルのインデクス
// 初期化されていないので、
// 直前にさしたカセットで書かれていた値が使われる
uint8_t rand_index;
uint8_t rand() {
    return RAND_TABLE[rand_index++];
}

// 技のひらめき境界値より小さい乱数を引けば技をひらめく
bool waza_hirameki() {
    return rand() < waza_threshold() ? true : false;
}
```

今日のまとめ

1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリの脆弱性を対象にしたアタック
 - Raw Hammer, Cold Boot Attack

出欠と感想

- 本日の講義でよくわかったところ, わからなかったところ, 質問, 感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので, そこにお願いします
 - パスワード : arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは来週の講義開始までに設定してあります
 - 仕様上「遅刻」表示になりますが, 特に減点等しません
 - 来週の講義開始までは感想や質問などを受け付けます

- 以下のスプレッドシートの自分の席のところに学籍番号を書いてください
 - https://docs.google.com/spreadsheets/d/1ctRdav7Mp5iSBTSMsTZePPcYLtP0ZtTodefkk6O2N1E/edit?usp=drive_link
 - 学生側から見た場合の配置で（画面上が教団側），席の対応する行と列のところに書いてください

感想とか質問の回答

- Ahead Pipelining の課題の緩和方法のところで、直前からの呼び出しも参照するようにするとあったが、二個前のアタリの予測が誤っていた場合二度手間になってしまうのではないかなと思いました。

- 1%の分岐予測ミスでも30%ほど性能を低下させてしまうというのは驚きだった。なぜこんなにも多くの分岐予測手法があるのかわかっていなかったが、今日の授業でその必要性や研究者たちの努力がわかった。

- RBiasに関する説明は最先端の研究の内容を日本語で聞くことができるなかなかない機会に非常に興味深かった。スライド75ページの内容の理解が難しかったのでもう少し説明していただけるとありがたかったと思う。

- Decoupled pipelineの大まかな動きはわかったような気がするが、例のパイプラインがあと1, 2段あれば確信が持てた気がする。

- スライドp.23色で強調された一つ目の00110パターンの少し右に、色で強調されていない00111パターンがあるので、テーブルの値は00ではなく01になると思いました（2bitであれば弱状態になるだけなのでどちらにせよ予測は0になるので結果に問題ないようですが）

- パーセプトロン予測器の学習のタイミングはいつなのでしょう？

- 現状パーセプトロン予測器は単層のものになっていると思うのですが、重みづけをもっと単純化することでパーセプトロンを複層に拡張した場合、さらなる予測器の精度向上は見込めるものなのでしょうか？

感想とか質問の回答

- 今回の講義では、TAGE や Hashed Perceptron などの高度な分岐予測器が特に印象に残った。最初は、複数の履歴長や tag、counter などが出てきてかなり複雑に感じたが、整理して考えると、TAGE は「どの程度長い履歴を見れば現在の分岐をうまく説明できるか」を探す仕組みだと理解できた。
- 特に面白いと感じたのは、TAGE が複数の履歴長の表を用意し、短い履歴では大まかな傾向を、長い履歴ではより具体的な文脈を捉えようとしている点である。この考え方は、複雑な信号を複数の尺度に分けて見るフーリエ展開や小波変換の考え方に少し似ているように感じた。ただし、TAGE は各成分を単純に足し合わせるのではなく、tag が一致した表の中から最も長い履歴を持つものを選ぶため、線形和というよりは「多尺度の文脈照合」に近いと感じた。
- 一方で、Hashed Perceptron や Statistical Corrector は、複数の表から得られた重みを加算して予測を行うため、こちらの方がより「複数の特徴を重み付きで統合する」方式に近いと理解した。TAGE が特定の履歴パターンを記憶して照合する方式であるのに対し、Perceptron 系は複数の弱い手がかりを組み合わせて判断する方式だと考えると、両者の違いが分かりやすくなった。

- BTBとOpCacheの説明がたりない

- パーセプトロン予測機の内積計算部分は足し算回路などが必要かと思うのですが1クロックで終わるような小さい回路をいれているのでしょうか？

- 講義で Ahead Pipelining の解説を聞いて、Zen 5 の 2-Ahead Branch Predictor を思い出し、関連する資料を読んでいたのですが、まだ十分には理解できていません。この両者は、どのような点で設計思想が似ており、またどのような点で異なるのでしょうか。

- グローバル分岐予測器のモチベーションで複数の分岐に相関があるものを拾いたいとのことですが、たとえば条件Xで必ずelse側を通るなら、後続の条件Yでも必ずelseを通る、といったような確率100%で発生する相関であればgccなどのコンパイラのオプション(-O3など)でまとめてしまうことはできないのでしょうか？ もちろんほとんどの場合は確率100%にはならないだろうし、仮にそうであるなら、そもそも人間側が高級言語でコーディングする際にまとめてしまえばいいという話になってしまいますが...

- 分岐予測器の学習や設計に用いられるコード（ベンチみたいな）みたいなものがあるのか気になった。

- 複数の分岐予測器を同時に走らせてアンサンブルさせるなどできるのかなと思った（速度の問題で多分使えなさそうだが）。

- 余談ですが、私はXORとXNORを混同して、どちらがどちらなのかを毎度30秒ほど考えるのですが、他の方にも共感してもらえるのか気になりました。

- パーセプトロンやTAGE周りの話でかなり理解に時間がかかってしまったので、家でも考えたいと思うのですが、求められる理解度はどれほどでしょうか？そういった技術の存在を知っている程度でよいのでしょうか？それとも、回路の意味なども含めて深く理解をしておいた方がよいのでしょうか？

- あと、分岐予測の日常的な例に困っていらっしゃるという話があったと思うのですが、「レジの店員が客の割り箸の必要性を予測するとして、客の購入品を見て、お弁当ならば用意しておく」みたいな感じでいかがでしょうか？

- 2006年に出たTAGE予測器が今も最先端を支えているのに驚きました。名古屋出身としては個人的に、名工大などの名古屋関係の大学がISCAのコンペで成果を出していると知って嬉しくなりました。RBiasで相関をとるときの計算コスト・時間コストがどうなっているか気になりました。

- 学部の授業でマイクロプロセッサを作った際、分岐予測の部分がクリティカルパス気味となりエントリ数を大きく取れなかったためにbimodal予測器がgshare予測器を上回ったのを思い出した。

- GPUみたいなSIMTアーキテクチャだと, warp divergenceが起こった後 32 threadのexecution pathがどうやって収束（合流？）するのか気になっている（間接分岐だとそこまで容易ではなさそう）

感想とか質問の回答

- 1. 資料41ページのPHTサイズの制約について、最近の対策を後で紹介する旨を仰っていましたが、キャッシュ周辺の話につながるのでしょうか。
- 2. 資料47ページのMPKI分布について：MPKIと予測精度の関係は、ベンチマークにおける分岐命令の割合に依存すると思います。そこで、一般のベンチマークでは、方向分岐/関節分岐/return の各分類の内訳はどういう場合が多いのか気になりました。

- 1-1. ハッシュ・パーセプトロンの効果：ターゲットの履歴より長いパターンで学習が進むのであれば、「（実際には5箇所だが）主に1箇所を更新」という状況に陥らないのか（資料61ページ目）
- 1-2. Ahead Pipelining：「インターリーブされている」とはどういうことか（資料89ページ目）

- 感想文の紹介がトピックごとに分類されているのが個人的には見やすいです。今後、さらに「質問→感想・コメント」のように分けていただくこともご検討いただけたりしますでしょうか。

- 分岐予測にパーセプトロンが使われているのは面白かった。この時代、機械学習からは逃れられないのかと感じた。

- 言語処理系を書いていると、至る所でパターンマッチによる条件分岐を書くが、何となくif文で書くよりは良さそうというくらいにしか考えていなかったもので、間接分岐命令というハードウェア実装が知れて面白かった。ノイズ除去の統計的アルゴリズムなどもそうですが、動的に改善されていくアルゴリズムは面白いです

- 2006年に開発(?)された予測器が20年たった今でも基盤となっていて、更新されていないのに驚いた。

- RBiasで、連続した分岐の1個目はミスが多い（この性質を利用している）とおっしゃていましたが、なぜミスが多いのですか。

- パーセプトロン式の予測器について、学習とかもハードワイヤされて(論理回路で組まれて)いるのか気になりました。自分はPythonでAdamなどの学習・最適化コードを書いたことがあるのですが、そういったものが実際にハードワイヤされているのですか？